University of Turin
School of Natural Sciences
Master's Degree in Computer Science

Track: Artificial Intelligence and Information Systems "Pietro Torasso"

Master's Thesis

Kolmogorov–Arnold Networks for Image Classification: An Experimental Study on Performance and Interpretability

First Supervisor
Prof. Attilio Fiandrotti

Second Supervisor
Prof. Roberto Esposito

Candidate
Leonardo Magliolo
Student ID: **844910**

Academic Year 2024/2025

In loving memory of Tino

Declaration of Originality

I declare to be responsible for the content I'm presenting in order to obtain the final degree, not to have plagiarized in all or part of, the work produced by others and having cited original sources in consistent way with current plagiarism regulations and copyright. I am also aware that in case of false declaration, I could incur in law penalties and my admission to final exam could be denied

Acknowledgments

I consider the completion of this journey to be the result of a collective effort on my part, for which I am deeply grateful. I wish to thank all my friends and family for understanding my situation, and for supporting me over the years—unconditionally, even in times of great uncertainty, in particular my mother, whose constant encouragement has always been a source of strength and inspiration.

I am indebted to Prof. Attilio Fiandrotti and Prof. Roberto Esposito, who have proven to be exceptional mentors not only academically but also personally. Their support extended well beyond the thesis itself, enabling me to obtain a doctoral fellowship abroad and to fulfill one of my greatest ambitions.

Finally, I would like to remember Tino, who passed away during this journey and left a lasting mark on me. Looking back, his generosity and moral example shaped the very best of what I received in my childhood, and I therefore dedicate this thesis to his memory.

Abstract

This thesis investigates Kolmogorov–Arnold Networks (KANs) for image classification, evaluating their performance and explainability against standard neural baselines. Under controlled, like-for-like conditions, we compare fully connected KANs with LeNet-300 on MNIST and a Convolutional KAN (ConvKAN) with LeNet-5 on EMNIST. Explainability is assessed using Grad-CAM and complementary attribution measures. Results show that KANs achieve accuracy comparable to conventional models but do not surpass them; LeNet variants retain a small, consistent advantage. XAI analyses reveal distinct evidence allocation: ConvKAN tends to produce sparser, more localized attributions, while LeNet distributes attention more broadly over class structure. Overall—KANs are competitive but not superior on MNIST/EMNIST, and XAI indicates complementary attention patterns.

Code Availability

All code used to run the experiments presented in this thesis is openly available at `this GitHub repository`. The repository contains training and evaluation scripts, model definitions, configuration files, and utilities for the explainability analyses.

Contents

Chapter 1

Introduction

This thesis compares architectures traditionally used in computer vision with their Kolmogorov–Arnold Network (KAN) counterparts. The goal is to evaluate whether shifting from node-based to edge-based parameterizations changes what models learn, how well they perform, and how transparently they can be analysed.

Research Questions

1. Under comparable conditions, do KAN counterparts match or surpass the predictive performance and data efficiency of conventional architectures?
2. When the same xAI tools are applied to both paradigms, do the resulting attribution maps reveal systematic differences in attention patterns?
3. Which quantitative measures most reliably capture alignment between model attributions and human-interpretable geometric cues, and do these measures consistently separate the two paradigms?
4. Are any observed differences stable across tasks and evaluation settings, or do they depend on specific problem characteristics?

1.1 Structure of the document

The thesis is organised into Chapters 2–8, each addressing a specific aspect of the problem.

- **Chapter 2** introduces neural networks and convolutional models. It covers foundational concepts such as learning rate, loss functions, backpropagation in multi-layer perceptrons, the universal approximation theorem, regularisation strategies, and the rationale for convolutional architectures. This chapter establishes the baseline knowledge required for the later comparisons.
- **Chapter 3** reviews the Kolmogorov–Arnold representation theorem and details the design of Kolmogorov–Arnold Networks (KANs). It discusses

critiques of the theorem, practical limitations of naïve implementations, and the modifications introduced to render KANs trainable and efficient. The architecture of KANs is described alongside their relationship to classical multi-layer perceptrons.

- **Chapter 4** focuses on spline functions within KANs. It defines spline functions, highlights their advantages over polynomials, and explains their role in parameterising KAN filters.

- **Chapter 5** provides an empirical comparison between LeNet300 and KaNet300 in fully connected settings. It describes the experimental protocol and hyperparameter choices, analyses learning dynamics (including principal-component analyses and spline evolution), and *reports performance results* with a detailed discussion of accuracy and loss.

- **Chapter 6** compares LeNet5 and KaNet5 in a convolutional setting. After describing the architecture setup, training protocol, and dataset, it presents results on accuracy and loss, analyses the impact of regularisation, and discusses the generalisation gap. This chapter forms the empirical core of the thesis from a performance perspective.

- **Chapter 7** is devoted to interpretability. It revisits Grad-CAM, explains how the method is adapted to KaNet5, and describes the extraction of feature maps. The chapter then presents class-specific heatmaps and statistical analyses of Grad-CAM activations under different normalisation schemes, introduces a dynamic visualisation system for exploring heatmap evolution, and defines structural attention metrics quantifying the overlap between activations and geometric primitives (edges, corners, loops, vertical strokes). It concludes with an aggregated assessment of sensitivity and selectivity at convergence.

- **Chapter 8 (Conclusions)** synthesises the findings of the thesis. It offers an overall assessment of the empirical comparisons (LeNet300/KaNet300 and LeNet5/KaNet5) and the interpretability analyses; consolidates the main contributions and outlines future research directions motivated by the results.

1.2 Main contributions

1. **Unified experimental framework, ablations, and results.** We conduct a head-to-head evaluation of fully connected (LeNet300 vs. KaNet300) and convolutional (LeNet5 vs. KaNet5) models on EMNIST under matched depth, width, pooling schedule, and optimisation settings. We also run *layer ablations*—OutputOnly, HiddenLayerOnly, Full—on the LeNet300/KaNet300 study to test whether the added functional complexity of KAN edge-wise

activations provides benefits as the number of hidden layers decreases. We report accuracy, loss, and generalisation behaviour across all variants.

2. **Class-discriminative analysis pipeline.** We specify and apply a consistent Grad-CAM workflow to KaNet5 and LeNet5, providing qualitative visualisations and quantitative summaries of the resulting heatmaps.

3. **Structural attention metrics and statistical aggregation.** We introduce geometry-aligned scores that combine normalised heatmaps with interpretable masks (edges, corners, loops, concavities, vertical-run concentrations) and aggregate results using non-parametric tests across normalisation regimes and epochs.

4. **Tools and insights for interpretability.** We deliver an interactive web interface to explore heatmap dynamics over training and distil evidence-based insights that separate *sensitivity* (amplitude) from *selectivity* (rank-based focus).

Taken together, these contributions advance our understanding of how classical and Kolmogorov–Arnold inspired architectures behave on vision tasks and provide new tools for their interpretation. They also highlight the importance of considering both absolute and relative activations when comparing models and suggest directions for future research.

Chapter 2

Neural Networks and Convolutional Models

This introductory chapter provides an accessible overview of the core ideas behind modern neural networks, with particular attention to concepts that recur throughout the thesis. We proceed from general principles to progressively more specialized models, maintaining a balance between intuition and mathematical precision. Where formulas are used, they are intentionally simple and accompanied by clear explanations.

2.1 Neural Networks in General

2.1.1 Basic Idea

Neural networks are flexible *representation learners* as much as they are function approximators. Given an input vector $\mathbf{x} \in \mathbb{R}^d$ (e.g., the pixel intensities of an image) and a desired output y (e.g., a class label), a network defines a parametric mapping $f_{\boldsymbol{\theta}} : \mathbb{R}^d \to \mathbb{R}^k$ whose parameters $\boldsymbol{\theta}$ (weights and biases) are learned from data. Crucially, this mapping is built by composing layers that transform the input into a sequence of *intermediate representations*:

$$\mathbf{h}^{(1)} = \sigma\Big(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\Big), \quad \mathbf{h}^{(2)} = \sigma\Big(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\Big), \ \ldots, \ \hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x}).$$

Each hidden vector $\mathbf{h}^{(\ell)}$ is not merely a computational by-product; it is a new description of the input where certain regularities become easier to exploit. Early layers typically re-express raw data into low-level features (e.g., edges or simple contrasts); middle layers consolidate these features into motifs; later layers organize motifs into task-relevant factors (e.g., object parts or class-specific cues). In this view, a layer exists to *reshape the geometry* of the data so that the final prediction can be made by simple operations (often linear) on a well-structured representation.

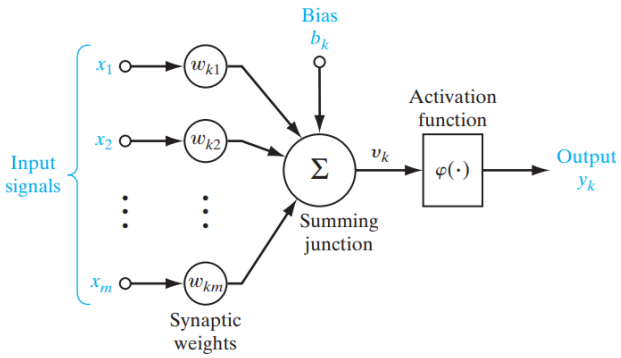


Figure 2.1: Diagram of a single perceptron. Input signals $x_1, \ldots, x_m$ are multiplied by synaptic weights $w_{k1}, \ldots, w_{km}$ and summed together with a bias b_k at a summing junction. The result is passed through a nonlinear activation function $\varphi(\cdot)$ to produce the output y_k. Source:[1], p. 43.

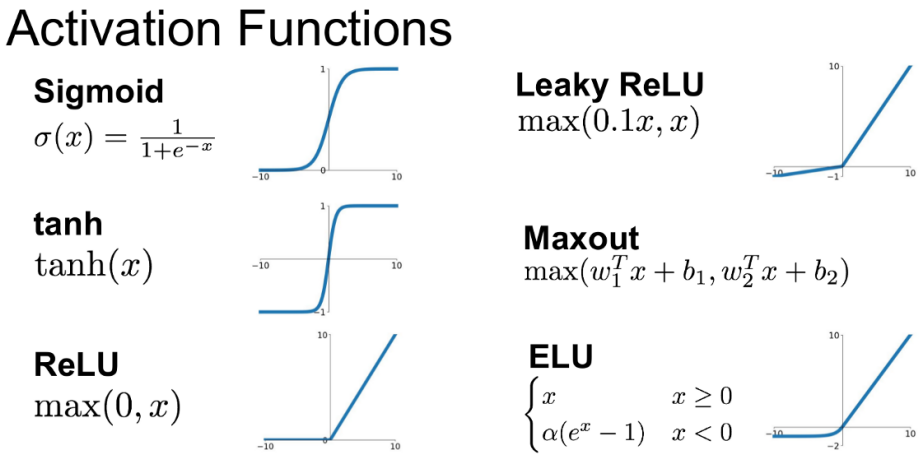


Figure 2.2: Comparison of common activation functions used in neural networks. The figure shows the shapes of the Sigmoid, Tanh, ReLU, Leaky ReLU, Maxout and ELU activations, highlighting differences such as saturation and slope. These activation functions play a crucial role in injecting non-linearity into the network and shaping the learned representations.

Source: Andrey Nikishaev, *How to debug neural networks. Manual*

Representations as feature spaces. One useful mental model is to think of each layer as defining a feature space in which classes are progressively more separable. If points from different classes are tangled in the original input space, successive non-linear transformations untangle them, creating clusters or directions that correspond to meaningful variations (such as stroke thickness or slant in handwriting). The parameters $W^{(\ell)}$ determine which variations are emphasized, while the activation $\sigma(\cdot)$ injects non-linearity so that new features need not be linear combinations of the old ones. By the time we reach the output, the representation is shaped so that a simple decision rule (e.g., taking an arg max over class scores) suffices.

A running example (digits). Consider classifying 28×28 grayscale images of handwritten digits into ten classes (0–9). Flattening the image yields $\mathbf{x} \in \mathbb{R}^{784}$, but the raw pixel space hides structure: two images of the same digit can be far apart if the stroke is shifted or thicker. Early layers learn detectors for primitive patterns (short strokes, endpoints, small curves) and encode them into $\mathbf{h}^{(1)}$. Subsequent layers pool and recombine these primitives into digit fragments (loops, crossings) within $\mathbf{h}^{(2)}$, and so on. The final layers operate on a representation in which different digits occupy distinct regions, enabling reliable classification. Although later chapters will employ convolutional layers to better capture spatial structure, the principle is identical for fully connected networks: learning is the progressive construction of task-aligned representations.

2.1.2 Learning Rate

Training adjusts $\boldsymbol{\theta}$ so that predictions $\hat{\mathbf{y}} = f_{\boldsymbol{\theta}}(\mathbf{x})$ match desired outputs while, at the same time, improving the quality of the intermediate representations learned by each layer. The standard workhorse is (stochastic) gradient descent, which updates parameters by stepping in the direction that *locally* most reduces the loss $\mathcal{L}(\boldsymbol{\theta})$:

$$\boldsymbol{\theta}_{t+1} \;=\; \boldsymbol{\theta}_t \;-\; \eta\, \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t),$$

where $\eta > 0$ is the *learning rate* and $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ is the gradient.

Why the negative gradient? The gradient points in the direction of *steepest increase* of the loss; therefore, taking a small step along the *negative* gradient moves us toward lower loss. Formally, using a first-order (Taylor) approximation around $\boldsymbol{\theta}_t$,

$$\mathcal{L}(\boldsymbol{\theta}_t + \Delta) \;\approx\; \mathcal{L}(\boldsymbol{\theta}_t) \;+\; \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)^{\top} \Delta.$$

Choosing $\Delta = -\eta\, \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$ yields

$$\mathcal{L}(\boldsymbol{\theta}_{t+1}) \;\approx\; \mathcal{L}(\boldsymbol{\theta}_t) \;-\; \eta \left\| \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t) \right\|_2^2 \;<\; \mathcal{L}(\boldsymbol{\theta}_t) \quad \text{for } \eta \text{ small enough.}$$

Thus, each update *minimizes the loss locally*: it reduces the current loss by an amount proportional to the squared gradient norm. In smooth regions (and with

sufficiently small η), this local decrease translates into a genuine descent step. Intuitively, as the loss decreases, earlier layers reshape their feature spaces so that downstream layers can separate classes more easily—representation learning and loss minimization happen together.

Full-batch vs. stochastic gradients. In practice we use mini-batches, replacing $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ with a noisy estimate $\widehat{\nabla}\mathcal{L}$ computed on a small subset of examples. Although each individual step may not reduce the loss, the estimator is (approximately) unbiased, so on average the updates still move in a descent direction. This stochasticity can even help escape shallow local minima or plateaus while continuing to refine useful features.

Role of the learning rate. The scalar η controls how aggressively we update not only the final classifier but the entire stack of learned features. If η is too large, the linear approximation above breaks down: steps may overshoot valleys, causing the loss to increase and useful features to be overwritten (oscillations or representation collapse). If η is too small, parameters—and thus the intermediate representations—change too slowly, requiring many iterations to meaningfully reshape the feature spaces. In practice, η is tuned empirically, often decreased during training via schedules (e.g., step or cosine decay) and combined with acceleration mechanisms such as momentum to stabilize and speed up the descent while preserving the beneficial evolution of representations.

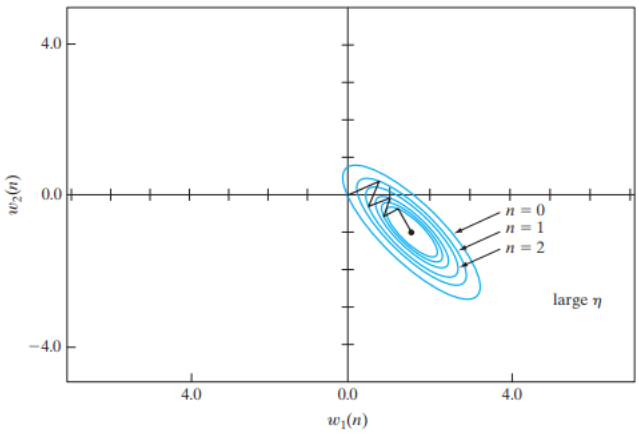


Figure 2.3: Effect of a large learning rate on gradient descent. Contour lines denote level sets of the loss function, and arrows show successive parameter updates ($n = 0, 1, 2$) in the (w_1, w_2) plane. A large step size causes overshooting across the valley, leading to oscillations rather than smooth descent. Source: [1], p. 128.

2.1.3 Loss Functions

The loss function quantifies the discrepancy between predictions and ground-truth labels over a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, and—importantly—acts as the *shaping signal* that sculpts intermediate representations. Two canonical choices illustrate this dual role:

- **Mean Squared Error (MSE)** for regression:

$$\mathcal{L}_{\text{MSE}}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N}\Big\|f_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i\Big\|_2^2.$$

  Minimizing MSE encourages representations that preserve information needed to predict a continuous target (e.g., a measurement), penalizing large deviations uniformly.

- **Cross-Entropy** for classification. For k classes with one-hot label $\mathbf{e}_{y_i}$ and predicted probabilities $\hat{\mathbf{p}}_i = \text{softmax}(\mathbf{z}_i)$ (where $\mathbf{z}_i$ are logits),

$$\mathcal{L}_{\text{CE}}(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{k}\Big[\mathbf{e}_{y_i}\Big]_c \log\Big(\hat{\mathbf{p}}_{i,c}\Big) \;=\; -\frac{1}{N}\sum_{i=1}^{N}\log\hat{\mathbf{p}}_{i,y_i}.$$

  Cross-entropy pairs naturally with the softmax because it rewards high probability on the correct class and penalizes mass on competitors.

In short, the loss does more than measure error—it *guides* how each layer should reshape the data. Effective training therefore hinges on a good match between the task, the loss, and the inductive biases of the architecture. Later sections will revisit how optimization choices interact with these losses to produce stable and discriminative intermediate representations.

2.1.4 Classification Tasks

In classification, the model outputs either a single probability (binary case) or a probability vector over classes (multiclass case). Decisions are typically made by $\arg\max_c \hat{\mathbf{p}}_c$. Performance is evaluated with accuracy, precision/recall, or more nuanced measures when class imbalance is present. In our digit example, a correctly trained model will place most probability mass on the true digit class and near-zero mass on others.

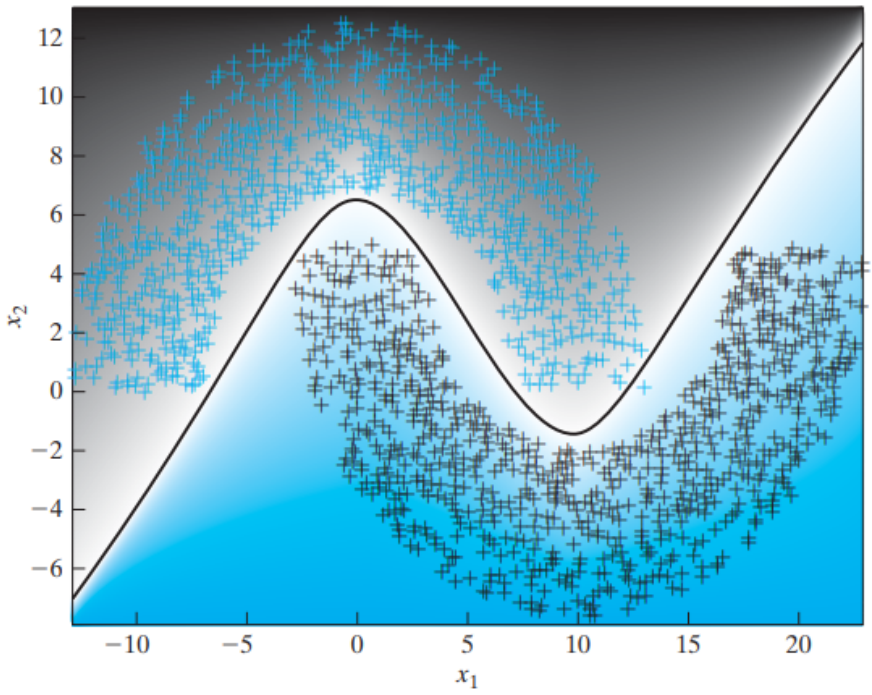


Figure 2.4: Illustration of a nonlinear classification problem. Two interleaved classes (blue crosses and black pluses) are separated by a complex boundary. This example underscores the need for learned representations and nonlinear transformations when classes are not linearly separable. Source:[1], p. 183.

2.2 Multi-Layer Perceptron (MLP)

An MLP is the canonical feed-forward network composed of fully connected (dense) layers with non-linear activations. Despite their simplicity, MLPs can model rich relationships and serve as a conceptual stepping stone to more specialized architectures.

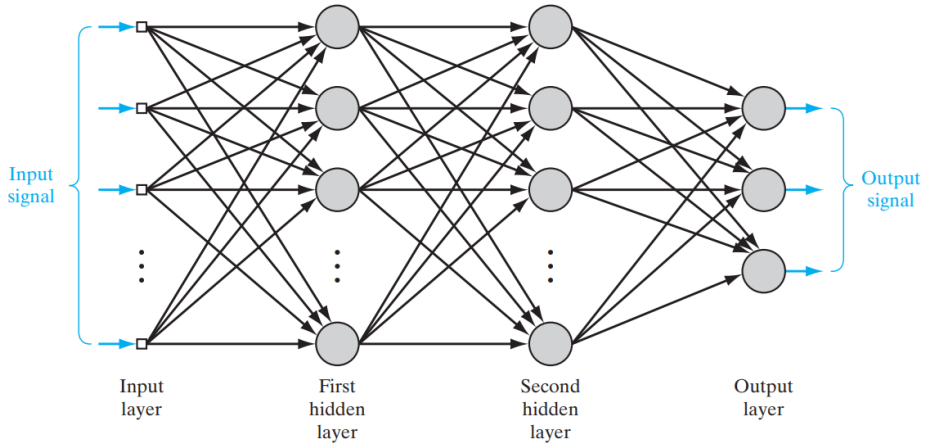


FIGURE 4.1 Architectural graph of a multilayer perceptron with two hidden layers.

Figure 2.5: Architectural graph of a multilayer perceptron with two hidden layers. The input signal enters at the left, traverses fully connected hidden layers where non-linear activations produce intermediate representations, and finally exits at the output layer. Such architectures can approximate complicated functions when trained appropriately. Source:[1], p. 155.

2.2.1 Backpropagation and the Need for Differentiable Losses

Training requires gradients of $\mathcal{L}$ with respect to *all* parameters across *all* layers. The backpropagation algorithm computes these gradients efficiently by applying the chain rule layer by layer from the output backward to the input. For a hidden layer with pre-activations $\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ and activations $\mathbf{h}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$, the local gradient has the form

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)}\left(\mathbf{h}^{(\ell-1)}\right)^{\top}, \quad \delta^{(\ell)} = \left(\left(W^{(\ell+1)}\right)^{\top}\delta^{(\ell+1)}\right) \odot \sigma'\left(\mathbf{z}^{(\ell)}\right),$$

where $\delta^{(\ell)} = \partial \mathcal{L}/\partial \mathbf{z}^{(\ell)}$ are the backpropagated errors and $\odot$ denotes elementwise multiplication. This recursion requires that the loss and the intermediate operations be differentiable (or subdifferentiable). In practice, common choices like ReLU are non-differentiable at a single point (zero) but admit subgradients, and cross-entropy is smooth, making backprop applicable.

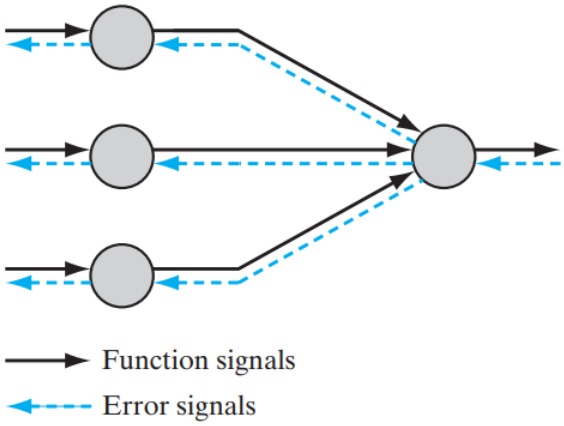


Figure 2.6: Schematic of the backpropagation algorithm. Solid arrows denote the forward propagation of function signals through layers, while dashed arrows denote the backward propagation of error signals used to compute gradients. Backpropagation efficiently computes derivatives for all parameters by reusing intermediate quantities. Source:[1], p. 156.

Implication. To train an MLP (or any deep network) with gradient-based methods, the loss $\mathcal{L}$ must be (sub)differentiable with respect to the parameters. This guides the choice of both the loss and the activation functions used in the network.

2.3 The Universal Approximation Theorem

The Universal Approximation Theorem (UAT) formalizes a central intuition: even a *single-hidden-layer* network with a suitable non-linear activation can approximate any continuous function on a compact domain to arbitrary accuracy, given sufficiently many hidden units [2]. More precisely, let σ be a non-polynomial, bounded and continuous activation (e.g., sigmoid). Then, for any continuous g on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and any $\varepsilon > 0$, there exists a network of the form

$$f(\mathbf{x}) = \sum_{j=1}^{m} \alpha_j \, \sigma\left(\mathbf{w}_j^\top \mathbf{x} + b_j\right) \quad \text{such that} \quad \sup_{\mathbf{x} \in \mathcal{K}} |f(\mathbf{x}) - g(\mathbf{x})| < \varepsilon.$$

Why it matters. The UAT assures us that feed-forward networks are expressive enough in principle. However, it is *existential*, not constructive: it does not specify how many units are needed, how to find the parameters, or how training behaves in finite data regimes. In practice, deeper networks with fewer units per layer often approximate complex functions more economically than a single very wide layer. This observation motivates deep architectures and specialized layers (such as convolutions) that embed useful inductive biases.

2.4 Regularization

2.4.1 Concept and Purpose

Regularization aims to improve generalization—the model's ability to perform well on new, unseen data—by discouraging overfitting to the training set. When a network is very flexible, it can memorize idiosyncrasies of the training data (including noise), leading to poor performance at test time. Regularization combats this by adding preference for 'simpler' solutions or by injecting noise that promotes robustness. Techniques range from architectural choices to data augmentation and explicit penalties on parameters. Here we focus on two widely used explicit penalties: ℓ_1 and ℓ_2.

2.4.2 ℓ_1 and ℓ_2 Regularization

Given a loss $\mathcal{L}_{\text{data}}(\boldsymbol{\theta})$ that measures data fit (e.g., cross-entropy), we define the regularized objective as

$$\mathcal{L}(\boldsymbol{\theta}) \;=\; \mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \lambda\,\Omega(\boldsymbol{\theta}),$$

where $\lambda \geq 0$ controls the strength of regularization and $\Omega(\boldsymbol{\theta})$ is a penalty on parameters. Two standard choices are:

$$\ell_2 \text{ (weight decay):} \quad \Omega(\boldsymbol{\theta}) = \tfrac{1}{2}\|\boldsymbol{\theta}\|_2^2 \;=\; \tfrac{1}{2}\sum_j \theta_j^2,$$
$$\ell_1 \text{ (sparsity):} \quad \Omega(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 \;=\; \sum_j |\theta_j|.$$

ℓ_2 effect. Adding $\frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$ yields a gradient contribution $\lambda\,\boldsymbol{\theta}$, causing parameters to shrink toward zero smoothly. This tends to distribute weights across features and is numerically stable. In many optimizers, *weight decay* directly implements this shrinkage per update.

ℓ_1 effect. The ℓ_1 penalty encourages exact zeros in parameters (sparsity). Intuitively, because the penalty grows linearly with $|\theta_j|$, small coefficients are strongly encouraged to drop to zero, promoting feature selection. The subgradient at zero is set-valued, but standard algorithms (e.g., proximal or coordinate methods) handle this; in deep learning practice, ℓ_1 is often used with care due to optimization sensitivity.

Choosing and tuning. The regularization strength λ is selected by validation; too large a value underfits (excessive bias), too small fails to curb variance. In the digit example, modest ℓ_2 often improves test accuracy, especially for small training sets. Additional regularizers (dropout, data augmentation, early stopping) can be complementary, but we emphasize ℓ_1/ℓ_2 here because they integrate directly with the objective function.

2.5 Momentum

Momentum is a modification to gradient descent that accelerates learning and reduces oscillations, especially in ravines or along directions with very different curvature scales. Instead of updating parameters purely from the current gradient, momentum maintains a velocity vector $\mathbf{v}_t$ that accumulates recent gradients:

$$\mathbf{v}_{t+1} = \beta\, \mathbf{v}_t + \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t),$$
$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\, \mathbf{v}_{t+1},$$

where $\beta \in [0, 1)$ is the momentum coefficient (e.g., 0.9). Intuitively, $\mathbf{v}_t$ is an exponentially weighted moving average of past gradients. When gradients consistently point in the same direction, momentum amplifies the effective step; when gradients oscillate, momentum damps the zig–zag.

2.5.1 Effectiveness

In ill-conditioned landscapes, plain gradient descent can make slow progress, taking tiny alternating steps across steep walls. Momentum smooths these directions, enabling larger effective moves along gentle valleys while limiting oscillations across steep directions. In practice, momentum often yields faster convergence and better final performance. It is a foundational idea behind many adaptive optimizers.

2.6 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are specialized neural architectures designed for data with spatial or spatiotemporal structure, such as images, audio spectrograms, or video frames. Unlike fully connected layers, which link every input to every neuron, a convolutional layer processes information through small learnable filters applied locally and repeatedly across the input domain. This design captures three essential principles: local receptive fields, parameter sharing, and translation equivariance.

Formally, let the input be $X \in \mathbb{R}^{H \times W \times C_{\text{in}}}$, where H and W denote height and width, and C_{in} the number of input channels. A convolutional layer is defined by a collection of kernels $K \in \mathbb{R}^{r \times r \times C_{\text{in}} \times C_{\text{out}}}$, with spatial size $r \times r$, spanning all input channels, and producing C_{out} output channels. The resulting feature maps $Y \in \mathbb{R}^{H' \times W' \times C_{\text{out}}}$ are given by

$$Y_{i,j,c} \;=\; \sum_{u=0}^{r-1} \sum_{v=0}^{r-1} \sum_{c'=0}^{C_{\text{in}}-1} K_{u,v,c',c}\; X_{i+u,\, j+v,\, c'} \;+\; b_c,$$

where (i, j) indexes the spatial position, c indexes the output channel, and b_c is a bias term.

In this formulation, each output value depends only on a local neighborhood of the input (the receptive field), the same kernel coefficients are reused across

all locations (weight sharing), and the operation is equivariant to translations of the input. By stacking multiple convolutional layers, CNNs progressively build hierarchical feature representations, from simple edges and textures to increasingly abstract and task-specific patterns.

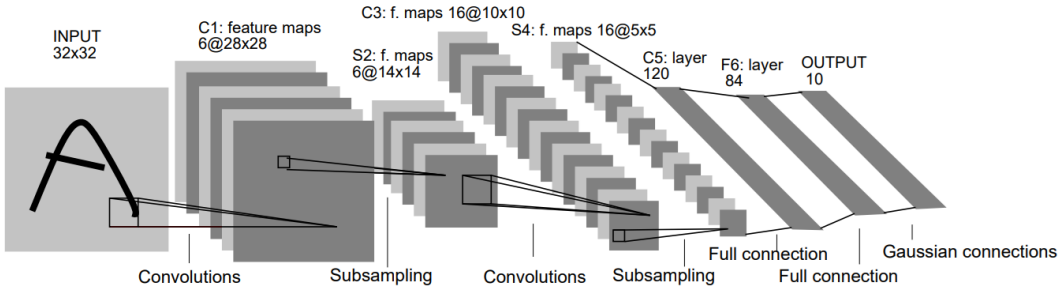


Figure 2.7: Architecture of LeNet-5, one of the earliest convolutional neural networks designed for handwritten digit recognition. The model alternates convolution and subsampling (pooling) layers, followed by fully connected layers and an output layer with Gaussian connections. Source:[3].

2.6.1 Why CNNs Often Outperform MLPs in Vision

MLPs treat every input dimension independently and must learn from scratch that nearby pixels in images are related. CNNs, by contrast, encode three key inductive biases:

(i) Local connectivity (local transformations). Each output depends on a small neighborhood in the input (the receptive field). This reflects the observation that visual patterns are locally coherent (edges, corners, textures). Locality reduces the number of parameters and focuses computation on meaningful interactions.

(ii) Weight sharing (shared weights). The same kernel is applied across the entire spatial domain. This enforces *translation equivariance*: shifting the input results in a proportionally shifted feature map. Weight sharing dramatically reduces parameters and data requirements, and it encourages features that detect patterns regardless of position (e.g., an edge anywhere in the image).

(iii) Hierarchical representations. Stacking convolutional layers yields a hierarchy: early layers detect simple primitives (edges/orientations), mid-level layers capture motifs (strokes, corners), and deeper layers assemble object parts and categories. Pooling or striding progressively enlarges the *effective* receptive field, enabling global understanding. This mirrors the compositional structure of images and leads to efficient, robust representations.

Further reading. For a rigorous, self-contained treatment that complements this overview, readers are encouraged to consult the comprehensive textbook [1]. Beyond clear derivations and geometric insights, it offers historical notes, worked examples, and end-of-chapter problems that make it a reliable companion for deeper study.

Chapter 3

Kolmogorov–Arnold Theorem and KAN Networks

3.0.1 Kolmogorov–Arnold representation theorem

The Kolmogorov–Arnold theorem (KAT) states that any continuous multivariate function defined on a compact domain can be expressed using only univariate functions and the operation of addition. In its classical formulation, for a function $f : [0,1]^n \to \mathbb{R}$ we have

$$f(x_1, \ldots, x_n) \;=\; \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \varphi_{q,p}(x_p) \right), \tag{3.1}$$

where $\varphi_{q,p} : [0,1] \to \mathbb{R}$ and $\Phi_q : \mathbb{R} \to \mathbb{R}$ are continuous univariate functions.

This result shows that addition is the only genuinely multivariate operation required: all the complexity of a multivariate function can be decomposed into combinations of univariate transformations of single coordinates and their sums.

Relevance for Kolmogorov–Arnold Networks. While the classical construction guarantees existence, the univariate functions it produces can be highly irregular, which historically limited its practical use. Kolmogorov–Arnold Networks (KANs) take the structural idea of (3.1) as an architectural blueprint: univariate functions are placed on *edges*, while *nodes* perform only summations. In the strict two-layer case, a KAN with width $2n+1$ and input dimension n directly mirrors the representation given in (3.1). In practice, however, KANs extend this principle by allowing deeper and wider compositions of learnable univariate functions, making the model both expressive and interpretable.

Representation via univariate functions and summation

Equation (3.1) can be interpreted as a two-step computational process built entirely from univariate functions and sums:

1. For each $q = 1, \ldots, 2n+1$, compute a coordinate-wise mapping and sum

$$s_q(x) = \sum_{p=1}^{n} \varphi_{q,p}(x_p).$$

2. Apply a univariate outer function to each s_q and aggregate:

$$f(x) = \sum_{q=1}^{2n+1} \Phi_q\Big(s_q(x)\Big).$$

This "edge-wise univariate function + node-wise summation" pattern is exactly the operational principle that KANs adopt. In this way, the Kolmogorov–Arnold theorem does not just provide a theoretical result, but also a concrete architectural template that motivates the design of KANs.

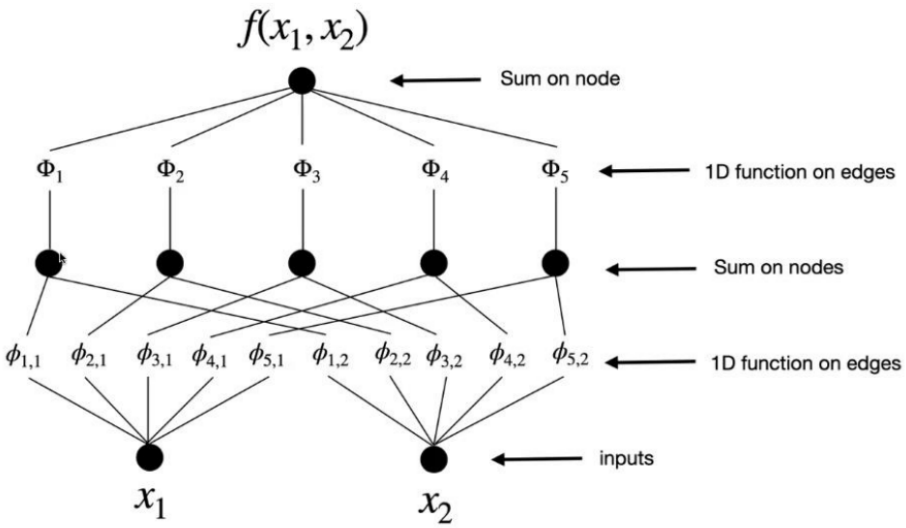


Figure 3.1: Kolmogorov–Arnold representation of $f(x_1, x_2)$ as sums of univariate functions on edges and nodes. Adapted from the presentation of the paper *Kolmogorov–Arnold Networks* [4].

3.0.2 Critiques and practical limitations

Why Kolmogorov's theorem is called "irrelevant" for ML (Poggio–Girosi). The Kolmogorov–Arnold theorem (KAT) is an *existence* result: it states that any continuous $f : [0, 1]^n \to \mathbb{R}$ can be represented as a finite superposition of univariate functions and sums. Poggio and Girosi [5] argue this is largely irrelevant for practical learning because (i) the theorem is nonconstructive and provides no *algorithm* to obtain the needed univariate components; (ii) the guaranteed decompositions can be highly irregular (non-smooth), which is incompatible with gradient-based training and with the smooth inductive biases used in practice; (iii)

it gives no approximation *rates*, no bounds on sample complexity, and no guidance on the size/conditioning of a trainable network; and (iv) it does not address *statistical* or *computational* efficiency (generalization, optimization dynamics, hardware fit). In short, KAT shows that a representation exists, but not that it is learnable, stable, efficient, or statistically sound.

On the "curse of dimensionality". KAT does *not* by itself defeat the curse. It replaces an n-variate problem with many 1D functions, but the *number* and *complexity* of these functions can still grow rapidly with n and with the target's complexity. Without additional structure (e.g., compositional sparsity, low-order interactions, smoothness), approximation rates remain dimension-dependent, and data requirements can still scale poorly. Thus, the theorem alone does not provide a generic cure for high-dimensional learning.

Positioning Kolmogorov–Arnold Networks (KANs). KANs adopt the KAT *template* (univariate maps on edges, sums at nodes) but modify it to address the above objections. First, edge maps are *smooth, low-dimensional* parametrizations (e.g., spline-based), which makes them trainable and regularizable. Second, KANs leverage *depth* to build intermediate smooth computations, avoiding the brittle, shallow factorizations suggested by the bare theorem. Third, training uses standard tools (normalization, regularization, pruning/distillation) to control complexity and improve identifiability. These choices do not turn KAT into a universal remedy for dimensionality, but they make the template *operational*: when the target exhibits low-dimensional interactions or compositional structure, KANs can be parameter-efficient and interpretable. Conversely, in unstructured, very high-dimensional regimes (e.g., raw perceptual data without symmetries), KANs typically need additional inductive biases or must be integrated with symmetry-aware modules.

Takeaway. Poggio and Girosi's critique holds for the classical theorem: existence $\neq$ learnability or efficiency. KANs are not a direct application of the theorem but a pragmatic reinterpretation: they keep the univariate-plus-sum *bias* while supplying trainable parameterizations, depth, and regularization. This preserves the conceptual link to KAT without relying on its impractical constructions, and it clarifies when such a bias is helpful and when it is not.

3.0.3 Solution proposed to mitigate practical limitations

The central objection to using the Kolmogorov–Arnold theorem (KAT) in machine learning is that it is an existence result devoid of constructive, smooth, and efficiently learnable components. The proposed solution is to turn the theorem into a practical inductive bias while supplying the missing ingredients that make it trainable, stable, and efficient.

From existence to learnability. Rather than relying on brittle two-layer factorizations, the approach embraces depth and represents each edge-wise mapping with a *residual* spline,

$$\varphi(x) = w_b\, b(x) + w_s\, \mathrm{spline}(x), \qquad b(x) = \mathrm{silu}(x),$$

initialized so that the spline term is near zero. This preserves well-scaled gradients at initialization and allows the spline to learn deviations only where needed. Since splines live on bounded grids while activations shift during training, knot grids are adapted on the fly to track the empirical activation range, preventing saturation and dead regions. These design choices address non-constructiveness and irregularity by replacing abstract univariates with smooth, low-dimensional modules that are compatible with gradient-based optimization.

Approximation and scaling. For deep KANs with k-th order B-spline activations, one obtains the bound

$$\left\| f - (\Phi_{L-1}^{G} \circ \cdots \circ \Phi_{0}^{G})x \right\|_{C^m} \;\le\; C\, G^{-(k+1)+m}, \qquad 0 \le m \le k,$$

where G is the grid size and C depends on the particular smooth KA representation. This yields a *dimension-independent* exponent $k{+}1$ (e.g., 4 for cubic splines) for the test error versus internal spline resolution, clarifying when KANs can effectively "beat" the curse of dimensionality—namely, when a smooth KA representation (possibly deeper/wider than the textbook $[n, 2n{+}1, 1]$) exists.

Accuracy vs. efficiency. To scale accuracy without restarting training, the *grid extension* procedure refines spline grids (coarse $\to$ fine) by reinitializing fine-grid coefficients to match the learned coarse spline, producing characteristic "staircase" drops in loss as G increases. Although per-edge functions introduce additional parameters, widths can remain small, yielding favorable accuracy–complexity trade-offs on structured targets.

Interpretability and model selection. A sparsification–pruning pipeline is introduced to discover compact subnetworks and enhance transparency: layerwise L^1 magnitudes quantify edge importance; an entropy term sharpens sparsity; nodes are pruned using incoming/outgoing scores; and salient edge maps can be *symbolified* by fitting simple analytic forms (e.g., sin, exp, log with affine wrappers). This addresses identifiability, simplifies models, and supports interactive inspection or distillation.

3.0.4 KAN architecture

Kolmogorov–Arnold Networks (KANs) realize the KAT template as a *composition of function matrices* whose entries are trainable univariate maps. Let $x_l \in \mathbb{R}^{n_l}$

denote the activations at layer l. Each node in layer $l{+}1$ sums edge-wise univariate transforms of the incoming coordinates:

$$x_{l+1,j} \;=\; \sum_{i=1}^{n_l} \varphi_{l,j,i}(x_{l,i})\,, \qquad j = 1, \ldots, n_{l+1}. \tag{3.2}$$

It is convenient to collect the edge functions into an *operator matrix*

$$\Phi_l \;=\; \begin{pmatrix} \varphi_{l,1,1}(\cdot) & \varphi_{l,1,2}(\cdot) & \cdots & \varphi_{l,1,n_l}(\cdot) \\ \varphi_{l,2,1}(\cdot) & \varphi_{l,2,2}(\cdot) & \cdots & \varphi_{l,2,n_l}(\cdot) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_{l,n_{l+1},1}(\cdot) & \varphi_{l,n_{l+1},2}(\cdot) & \cdots & \varphi_{l,n_{l+1},n_l}(\cdot) \end{pmatrix}, \qquad x_{l+1} \;=\; \Phi_l\, x_l, \tag{3.3}$$

where the "matrix–vector product" is understood exactly as in (3.2): apply $\varphi_{l,j,i}$ to the i-th component of x_l and sum over i. A depth-L KAN with shape $[n_0, n_1, \ldots, n_L]$ is the composition

$$\mathrm{KAN}(x) \;=\; (\Phi_{L-1} \circ \Phi_{L-2} \circ \cdots \circ \Phi_0)\, x, \qquad x \in \mathbb{R}^{n_0}. \tag{3.4}$$

Capacity control and complexity. KANs expose two complementary levers of expressivity. *External* degrees of freedom are determined by the graph shape $[n_0, \ldots, n_L]$ (depth/width). *Internal* degrees of freedom live inside the entries of Φ_l, which are one-dimensional modules whose resolution is set by spline order k and grid size G (details of their residual implementation and adaptive grids are given in Section 3.0.3, not repeated here). For a depth-L, width-N network with k-th order splines on G intervals, the parameter count scales as $O(N^2L(G{+}k)) \approx O(N^2LG)$.

Training and simplification (pointer). All operations in (3.2)–(3.4) are differentiable and trained end-to-end. Procedures for grid extension, sparsification, pruning, and symbolification are part of the training pipeline and are described in Section 3.0.3.

In essence, KANs are precisely the operators in (3.3) stacked as in (3.4): nodes perform summation; edges carry the learnable univariate maps. This matrix view captures the architecture without duplicating the optimization details handled in Section 3.0.3.

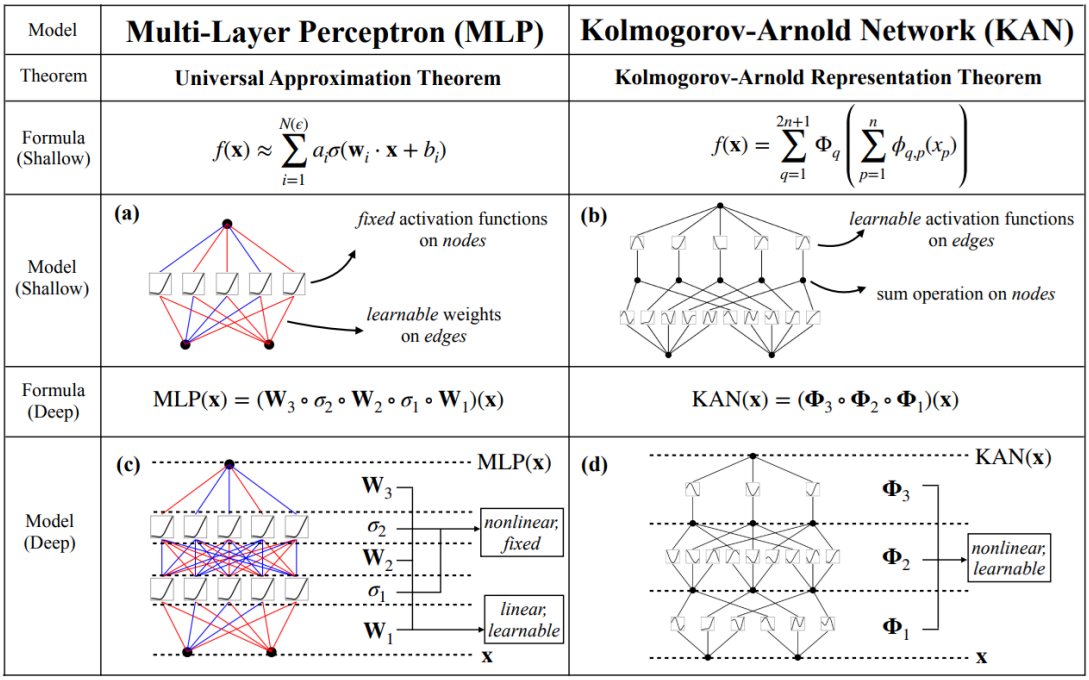


Figure 3.2: Comparison between Multi-Layer Perceptrons (MLPs) and Kolmogorov–Arnold Networks (KANs). In MLPs the nonlinearities are fixed at the nodes and edges carry scalar weights, while in KANs nonlinearities are learnable univariate functions on edges and nodes only perform summation. The diagram highlights the shallow and deep formulations of both architectures. Adapted from [4], Fig. 0.1.

Further reading. For a comprehensive and authoritative treatment of the Kolmogorov–Arnold representation theorem and its reinterpretation as a modern neural architecture, readers are encouraged to consult [4]. In addition to introducing the original motivation and design of Kolmogorov–Arnold Networks, it provides theoretical results, implementation details, and extensive empirical benchmarks that complement the overview presented in this chapter.

Chapter 4

Spline Functions in KAN

4.1 Definition of spline functions

The construction of spline functions can be understood as a natural progression, starting from the simplest form of interpolation between two points, moving to polynomial curves defined by sets of control points, and culminating in piecewise-defined functions with controllable smoothness. We outline this path step by step.

Linear interpolation ("lerp"). The most elementary form of interpolation is the *linear interpolation*, or "lerp". Given two points $A, B \in \mathbb{R}^d$ and a parameter $t \in [0, 1]$, the interpolated point is defined as

$$\operatorname{lerp}(A, B; t) \;=\; (1 - t)\, A + t\, B.$$

Geometrically, this formula describes the straight line segment connecting A and B, parametrized by t. At $t = 0$ we recover A, at $t = 1$ we obtain B, and for intermediate values the expression lies strictly inside the convex hull of $\{A, B\}$.

Bézier curves via repeated lerp (de Casteljau recursion). By repeatedly applying lerp, one obtains Bézier curves. Given $n+1$ control points $P_0, \ldots, P_n \in \mathbb{R}^d$, one defines the de Casteljau scheme:

$$\begin{aligned} B_i^{(0)}(t) &= P_i, && i = 0, \ldots, n, \\ B_i^{(k)}(t) &= (1 - t)\, B_i^{(k-1)}(t) + t\, B_{i+1}^{(k-1)}(t), && i = 0, \ldots, n - k. \end{aligned}$$

The curve is then given by

$$\mathbf{b}_n(t) \;=\; B_0^{(n)}(t), \qquad t \in [0, 1].$$

This recursive construction emphasizes that Bézier curves are nothing more than nested linear interpolations. Figure 4.2 illustrates this idea for the quadratic case: intermediate points are first interpolated, then interpolated again, producing a smooth quadratic segment.

Alternatively, one can write $\mathbf{b}_n$ in closed form using the Bernstein polynomials:

$$\mathbf{b}_n(t) \;=\; \sum_{i=0}^{n} \beta_{i,n}(t)\, P_i, \qquad \beta_{i,n}(t) \;=\; \binom{n}{i} (1-t)^{n-i} t^i.$$

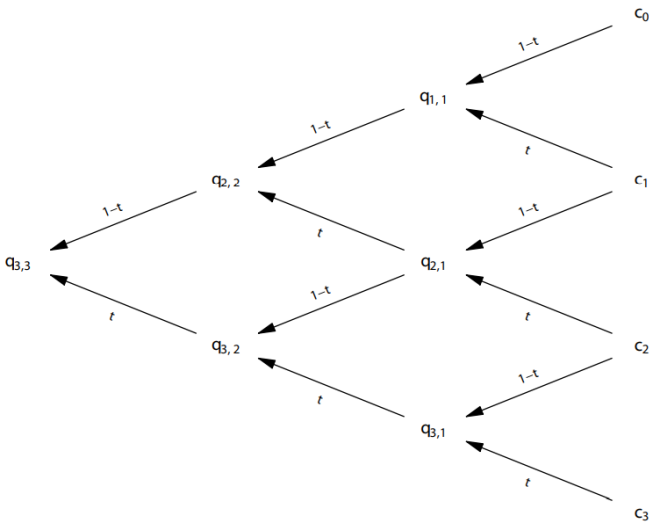


Figure 4.1: De Casteljau scheme for the computation of a cubic Bézier curve. At each stage, new points are obtained by linear interpolation of the previous ones, until the final curve point is determined. Adapted from [8], Fig. 1.7.

Sum of Bézier coefficients (partition of unity). The Bernstein polynomials $\beta_{i,n}(t)$ have the crucial property that they form a *partition of unity*:

$$\sum_{i=0}^{n} \beta_{i,n}(t) \;=\; \sum_{i=0}^{n} \binom{n}{i} (1-t)^{n-i} t^i \;=\; \Big((1-t)+t\Big)^n \;=\; 1, \qquad \forall\, t \in [0,1].$$

Hence $\mathbf{b}_n(t)$ is always a convex combination of the control points, and lies inside their convex hull.

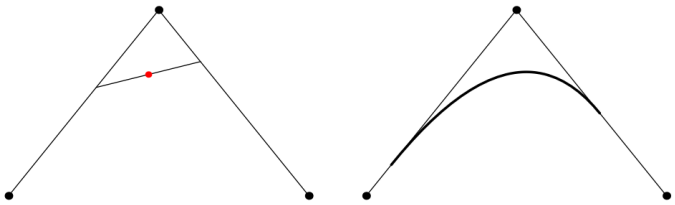


Figure 4.2: Construction of a quadratic Bézier (spline) segment. The red point is obtained as an intermediate interpolation, and the resulting curve smoothly interpolates the endpoints. Adapted from [8], Fig. 1.18.

Spline functions. Bézier curves, while flexible, are defined globally by all control points at once. For many applications—such as computer graphics, geometric modeling, or machine learning—it is more advantageous to work with *piecewise polynomial* functions, which allow local control of shape. This leads naturally to the notion of splines.

Let $\Xi = \{a = \tau_0 \leq \tau_1 \leq \cdots \leq \tau_m = b\}$ be a nondecreasing sequence of real numbers called *knots*, and let $k \in \mathbb{N}$ be the polynomial degree. A (univariate) spline $s : [a, b] \to \mathbb{R}$ of degree k with knots Ξ is defined by:

- on each interval $[\tau_j, \tau_{j+1})$, s coincides with a polynomial of degree at most k;
- at the interior knots, s satisfies prescribed smoothness conditions, typically being $s \in C^r$ with $0 \leq r \leq k$, depending on the multiplicity of the knots.

Thus, a spline is a continuous function built from polynomial pieces, smoothly joined at specified points. The vector space of all such splines, denoted $S_k(\Xi)$, admits particularly convenient bases. Among these, *B-splines* stand out due to their locality, nonnegativity, and partition of unity property.

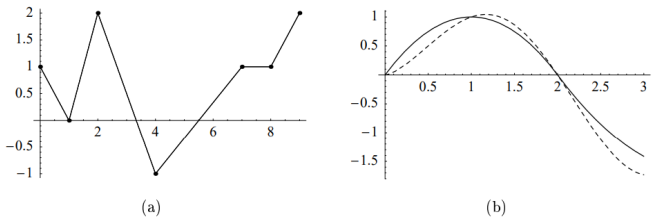


Figure 4.3: (a) A sequence of data points joined by linear interpolation (*lerp*), producing a piecewise linear function. (b) Comparison between linear interpolation (dashed line) and quadratic spline interpolation (solid line), highlighting the smoother behavior of higher-order constructions. Adapted from [8], Fig. 2.5.

4.1.1 Runge phenomenon and polynomial fitting limits

Global polynomial interpolation often behaves counterintuitively: increasing the degree does not always improve the fit and may even create large oscillations, especially near the interval boundaries. This effect is known as the *Runge phenomenon*. In what follows we explain it with simple formulas and two visual examples.

Setup (interpolation on $[-1, 1]$). Given a function $f : [-1, 1] \to \mathbb{R}$ and distinct nodes $x_0, \ldots, x_n \in [-1, 1]$, the degree-$n$ interpolant p_n is the unique polynomial satisfying

$$p_n(x_i) = f(x_i), \qquad i = 0, \ldots, n.$$

Using the Lagrange basis $\{\ell_i\}_{i=0}^n$, one can write

$$p_n(x) = \sum_{i=0}^{n} f(x_i)\,\ell_i(x), \qquad \ell_i(x) = \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{x - x_j}{x_i - x_j}.$$

These formulas are simple and exact, but they already hint at possible instability: the factors $\frac{x-x_j}{x_i-x_j}$ can become large when x is near the ends of the interval and the nodes are evenly spaced.

A canonical example (Runge's function). Consider the smooth function

$$f(x) = \frac{1}{1 + 25x^2}, \qquad x \in [-1, 1].$$

If we take *equispaced* nodes and increase n, the interpolant p_n develops big oscillations near $x = \pm 1$. Figure 4.4 shows this behavior: low degrees underfit (too flat), moderate degrees look reasonable, and higher degrees wiggle and overshoot at the boundaries. This is the Runge phenomenon in a nutshell.

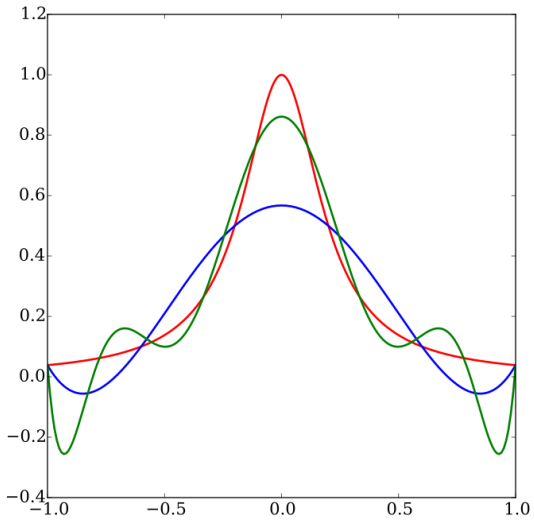


Figure 4.4: Interpolation of Runge's function $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$ using equispaced nodes. As the degree grows, the interpolant (colors) exhibits large endpoint oscillations despite f being smooth. Source: *Wikipedia*, `https://en.wikipedia.org/wiki/Runge%27s_phenomenon`.

Why it happens A concise way to see the problem is to examine the *nodal polynomial*

$$\omega_{n+1}(x) = \prod_{i=0}^{n} (x - x_i),$$

which vanishes at all nodes. Near the endpoints, $|\omega_{n+1}(x)|$ becomes very large for equispaced nodes, and the Lagrange basis functions $\ell_i(x)$ (ratios of such products) inherit this growth. Intuitively, the interpolant must bend wildly to hit many closely packed endpoint nodes, creating oscillations. Choosing *Chebyshev* nodes (clustered near ± 1) mitigates this growth and greatly reduces oscillations, but the fit remains a single global object: a local change in data alters the entire polynomial.

Practical limits of global polynomial fitting. The next experiment (Figure 4.5) shows noisy data (blue dots) fitted by polynomials of degrees 1 (red), 2 (blue dashed), and 300 (green). The linear and quadratic fits capture the trend without oscillations, while the very high-degree fit chases noise, oscillates, and explodes near the edges. This illustrates three practical limits:

1. *Global support*: every coefficient affects the whole interval.
2. *Noise amplification*: high degrees overfit and oscillate.
3. *Endpoint instability*: oscillations concentrate near the boundaries.

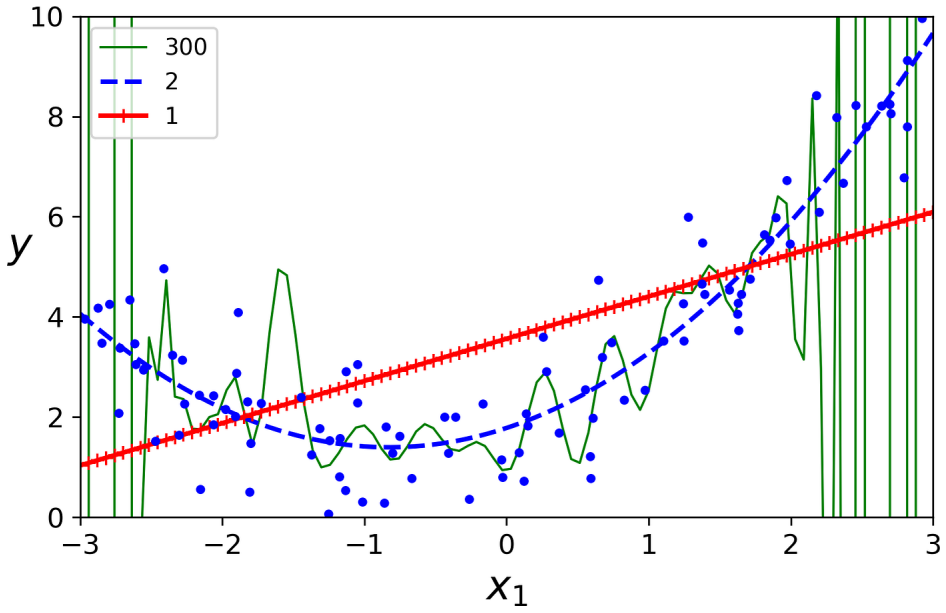


Figure 4.5: Polynomial regression on noisy data. Degree 1 (red) and 2 (blue dashed) give stable trends; degree 300 (green) overfits and shows pronounced endpoint oscillations reminiscent of Runge's phenomenon. Source: Yonggeun Shin, `https://yganalyst.github.io/ml/ML_chap3-3`.

4.1.2 Advantages of B-splines

B-splines combine geometric intuition with numerical robustness that global polynomials typically lack. We summarize the key points concisely.

Local basis with minimal support. Let $\{N_{j,k}\}_j$ be the degree-k B-spline basis on a knot vector Ξ. Every spline $s \in S_k(\Xi)$ admits the local expansion

$$s(x) = \sum_j c_j \, N_{j,k}(x),$$

and each $N_{j,k}$ is nonzero on at most $k+1$ consecutive knot spans. A single coefficient c_j therefore influences s only locally, improving conditioning and interpretability.

Partition of unity and convex-hull bounds. B-splines satisfy $N_{j,k}(x) \geq 0$ and $\sum_j N_{j,k}(x) = 1$ for all x, so

$$\min_j c_j \;\leq\; s(x) \;\leq\; \max_j c_j.$$

This guards against spurious overshoots on each local support and makes $\{c_j\}$ true control values.

Continuity and efficient evaluation. With simple interior knots, degree-k splines are C^{k-1}; a knot of multiplicity r reduces continuity to C^{k-r}. Evaluation is stable via the Cox–de Boor recursion

$$N_{j,0}(x) = \mathbf{1}_{[\tau_j, \tau_{j+1})}(x), \qquad N_{j,k}(x) = \frac{x - \tau_j}{\tau_{j+k} - \tau_j} N_{j,k-1}(x) + \frac{\tau_{j+k+1} - x}{\tau_{j+k+1} - \tau_{j+1}} N_{j+1,k-1}(x),$$

and at any x at most $k+1$ basis functions are active, yielding $O(k)$ cost for values and derivatives.

Local refinement and robustness to endpoint oscillations. Accuracy is increased by inserting knots (reducing the local mesh size) rather than by raising a global degree; this piecewise low-degree construction is less susceptible to Runge-type oscillations near the interval boundaries than high-degree global polynomials.

Computational advantage over global polynomials. For samples $\{x_r\}_{r=1}^N$, the collocation matrix $A_{rj} = N_{j,k}(x_r)$ has

$$\operatorname{nnz}(A) = O\Big(N(k+1)\Big),$$

and $A^\top A$ is banded with bandwidth $\leq 2k$. A degree-n polynomial fit instead uses the dense Vandermonde matrix $V_{ri} = x_r^i$ with

$$\operatorname{nnz}(V) = O\Big(N(n+1)\Big),$$

and $V^\top V$ is dense and often ill-conditioned for equispaced nodes. Splines thus deliver sparse algebra and better numerical stability.

Implications for KAN. In Kolmogorov–Arnold Networks, each edge-wise activation is a univariate B-spline,

$$\varphi(x) = \sum_{i=0}^{G-1} c_i \, B_i^{(k)}(x),$$

so only $k+1$ coefficients are active at any input, enabling $O(k)$ forward/backward cost per edge. Capacity grows by *grid extension* (knot insertion), which refines locally without altering the global shape.

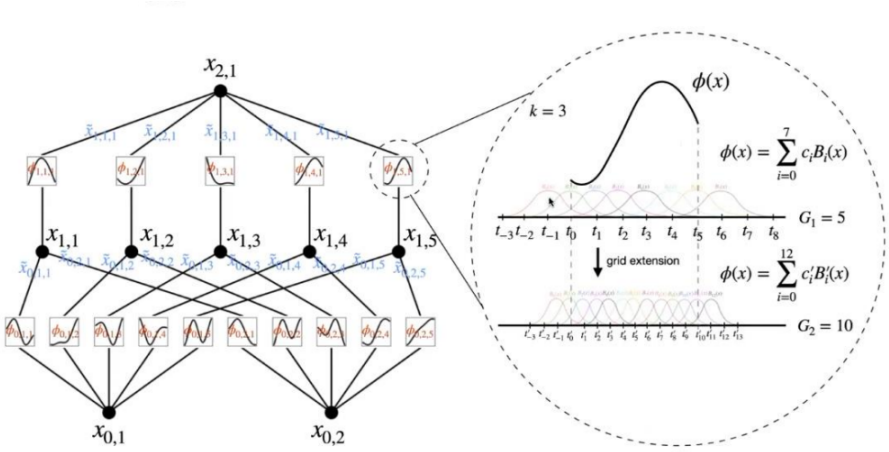


Figure 4.6: B-splines in KAN. Each edge activation $\varphi(x)$ is a degree-k B-spline; increasing the grid size ($G_1 \to G_2$) via knot insertion adds local resolution without changing the global shape. Adapted from [4], Fig. 2.2.

Summary. Local support, partition of unity, tunable smoothness, $O(k)$ evaluation, and sparse banded systems make B-splines stable and efficient—well suited as learnable, edge-wise activations in KAN.

Further reading. For a systematic introduction to the mathematical foundations of spline theory, readers are encouraged to consult [8], which covers Bézier curves, B-splines, approximation theory, and computational aspects in depth. For their application as trainable activation functions in Kolmogorov–Arnold Networks, the reference [4] provides a detailed account of the architectural motivations, implementation choices, and empirical benefits. Together, these works complement the overview presented here by connecting classical spline theory with its modern use in neural network design.

Chapter 5

Connected KAN Architectures: Experimental Results and Insights

This chapter compares the classical multilayer perceptron **LeNet300** with its Kolmogorov–Arnold inspired counterpart **KaNet300**. Across all ablation variants on MNIST, the two models achieve *almost parity* in terms of accuracy and loss. LeNet300 shows marginally more stable training dynamics, but KaNet300 does not provide a systematic advantage. Overall, the results suggest that both families perform on essentially the same level within this experimental setting.

5.0.1 Architecture descriptions

LeNet300 configurations

The LeNet-300 family represents a standard multilayer perceptron (MLP) backbone for image classification tasks on MNIST-like data. Inputs of size 28×28 are first flattened into a vector of dimension 784, which is then processed by one or more fully connected layers with ReLU activations. A final linear layer projects the hidden representation to 10 output logits.

Three architectural variants are considered:

- **LeNet300_OutputOnly**: a direct linear projection from input to output ($784 \to 10$).
- **LeNet300_HiddenLayerOnly**: a single hidden layer of width 300, followed by the output layer ($784 \to 300 \to 10$).
- **LeNet300_Full**: two hidden layers of widths 300 and 100, followed by the output layer ($784 \to 300 \to 100 \to 10$).

In all cases, nonlinearities are fixed and placed on the *nodes*, while the connections between layers are simple linear weight matrices. This conventional design provides the baseline against which KAN architectures are compared.

Kanet300 variants

The Kanet300 family mirrors the dimensional structure of LeNet-300, but replaces its linear-weight connections with learnable univariate functions. In Kolmogorov–Arnold Networks (KANs), each *edge* carries a nonlinear function, while nodes simply sum their incoming signals. This design shifts the role of nonlinearity from nodes to edges.

Each connection is parameterized by a spline-based function, ensuring smooth and data-adaptive transformations. The output of a node is thus the sum of its incoming edge-functions, possibly followed by a simple outer activation. In our implementations, the chosen pre-activation function is $ReLU$. This specific choice was made to render the comparison with LeNet-300 as fair as possible, since both families share the same baseline nonlinearity.

Three architectural variants parallel those of LeNet-300:

- **Kanet300_OutputOnly**: a direct KAN layer mapping inputs to outputs ($784 \to 10$) with nonlinear edge functions.
- **Kanet300_HiddenLayerOnly**: one hidden KAN layer of width 300, followed by a KAN output layer ($784 \to 300 \to 10$).
- **Kanet300_Full**: two hidden KAN layers of widths 300 and 100, followed by a KAN output layer ($784 \to 300 \to 100 \to 10$).

The Kanet300 family is therefore structurally aligned with LeNet-300, but differs fundamentally in the placement and nature of nonlinearities: *node activations vs. edge functions*. This distinction highlights the Kolmogorov–Arnold inspired reallocation of expressivity within the architecture.

5.0.2 Preliminary study on *LeNet300* and *KaNet300*

PCA study

This exploratory analysis was conducted *exclusively* on *LeNet300* models and was designed to assess the contribution of each normalization type (L1, L2, none) across the three ablation configurations (*OutputOnly*, *HiddenLayerOnly*, *Full*). Our goal was to see whether peculiar or interesting structures emerge in the PCA space spanned by the first three components (PC1–PC3), and how these structures evolve across epochs.

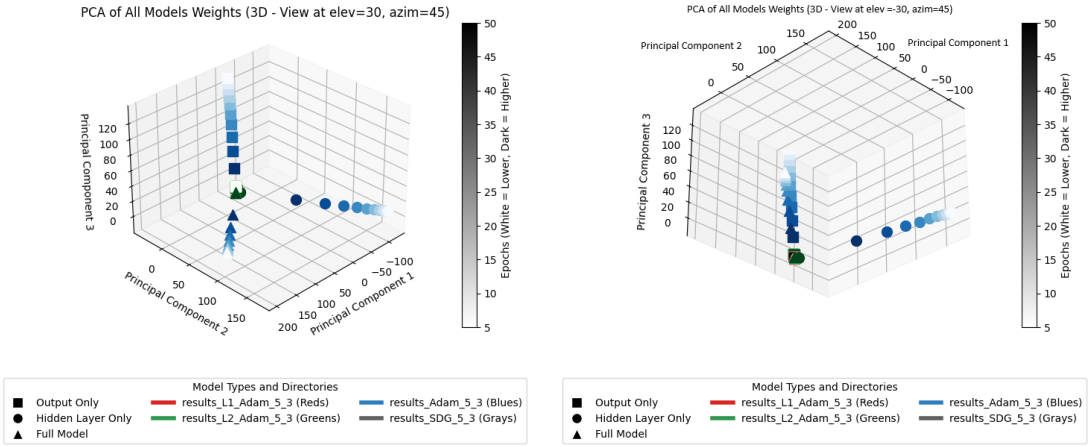


Figure 5.1: Comparison of PCA projections of LeNet300 weights across ablation settings, optimizers, and normalization schemes. Color encodes training epochs, while marker shapes denote architecture variants.

What the coordinates suggests

- *Adam without regularization stands out.* Runs with `reg=none` and `opt=Adam` travel very long trajectories in PCA space as epochs increase, and they do so in *architecture-specific directions*. In *Full* the drift is mostly along PC1 and PC2; in *HiddenOnly* PC2 grows while PC1 decreases; in *OutputOnly* the movement is dominated by PC3. Visually, these three families separate markedly from all other runs.

- *With L1/L2 (Adam) and with SGD (none), the embeddings cluster tightly.* These groups occupy a compact region and show modest, smooth drift with epoch. Most of this progression is aligned with a single axis (often PC2), giving the impression of a clean temporal ordering without large detours.

- *Epoch as a "direction" in the embedding.* For the regularized (L1/L2) Adam runs, PC2 tends to change monotonically with epoch (typically decreasing), providing a visually consistent timeline in the PCA plot. By contrast, the `none` groups diverge: in *Full* and *HiddenOnly* they move in the opposite direction along PC2, while in *OutputOnly* the evolution is largely captured by PC3 instead.

- *Regularization strength matters.* L2-regularized Adam runs tend to traverse slightly larger distances in the embedding than L1-regularized ones (still far smaller than the unregularized Adam case), suggesting a somewhat stronger reshaping of representations over time.

- *Ablation configurations imprint different geometries.* Even within the same optimizer/regularization, *OutputOnly*, *HiddenOnly*, and *Full* leave distinct

geometric signatures in PCA space, though for the regularized settings these differences remain subtle and largely co-located.

Practical implications from PCA analysis

The principal component analysis of *LeNet300* runs provides several insights into the training dynamics on MNIST:

- Models trained with **Adam without regularization** drift far across the PCA space, following divergent trajectories as epochs increase. This suggests less stable training dynamics and potentially less reliable representations.
- With **Adam+L1/L2** or with **SGD**, trajectories remain compact and orderly, progressing smoothly along a main axis. This indicates more stable and predictable convergence.
- The three ablation setups (*OutputOnly*, *HiddenOnly*, *Full*) exhibit **distinct PCA geometries**, but with regularization these differences become subtle; without it, they are amplified, underscoring the role of normalization in constraining training evolution.

Spline evolution

To better understand how KAN activation functions evolve during training, we visualize the spline transformations across consecutive epochs. In the plots below, the blue curve represents the spline at the current epoch, while the green curve shows the spline at the following epoch. This pairwise comparison highlights how the learned activation shapes are refined as training progresses.

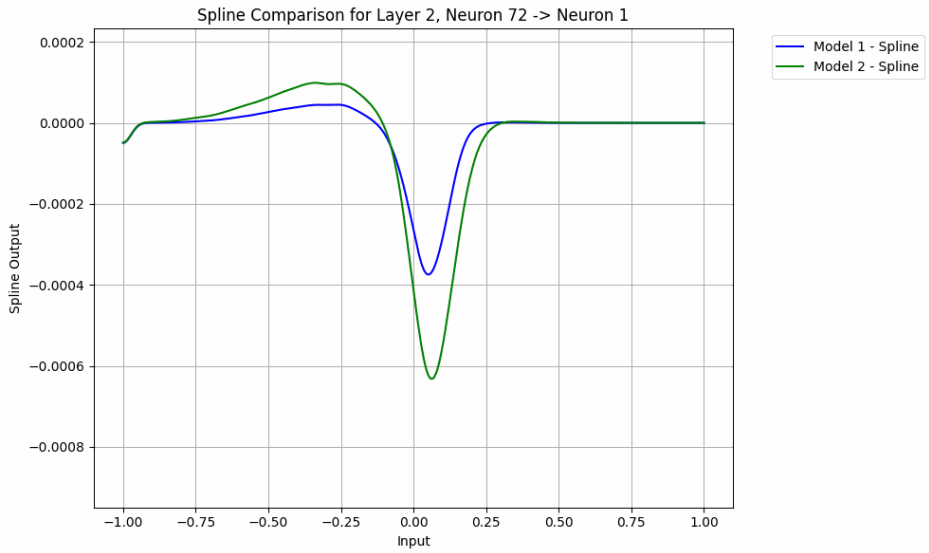


Figure 5.2: Spline comparison between epoch 1 and epoch 2. Most changes occur during the very early stages of training.

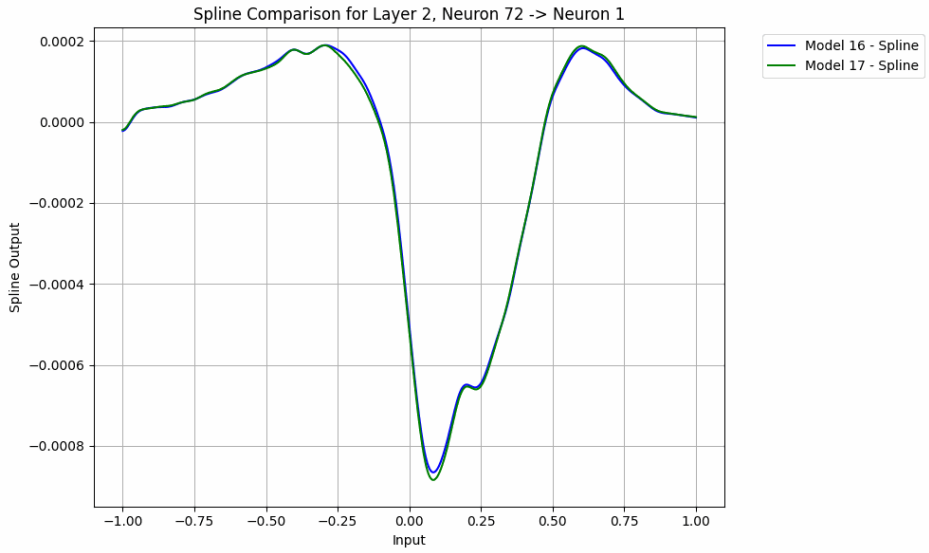


Figure 5.3: Spline comparison between epoch 16 and epoch 17. At this stage, the spline is already close to stabilization, with only moderate adjustments.

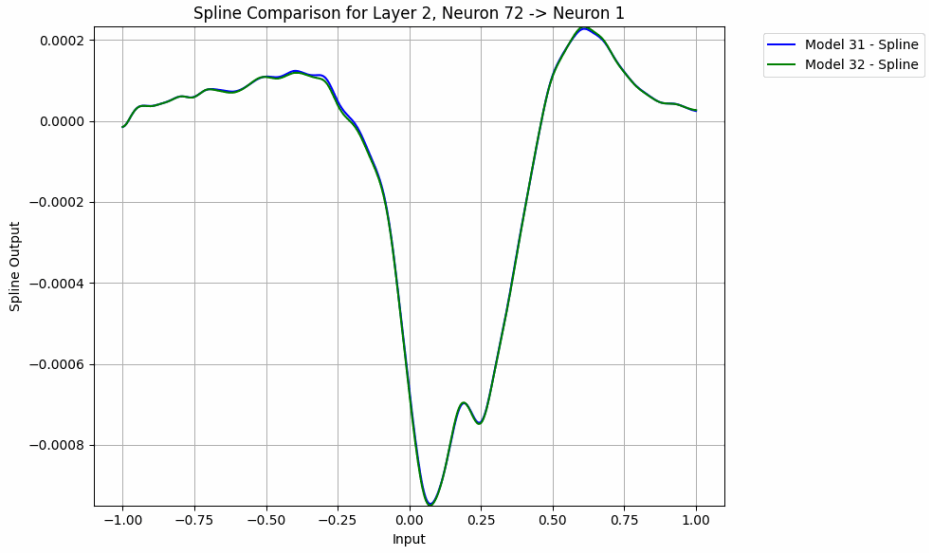


Figure 5.4: Spline comparison between epoch 31 and epoch 32. Only marginal refinements are visible, confirming that most changes happen within the first few epochs.

A qualitative inspection of several neurons across different layers confirms that the majority of spline adjustments occur within the first five epochs. Later epochs contribute only minor refinements, suggesting that spline-based activations in KANs converge quickly to their effective functional shape.

5.0.3 Experimental protocol

Hyperparameters and optimizer

We evaluated both adaptive and non-adaptive first-order methods, namely *Adam* and *SGD with momentum*. Generally, Adam tended to converge rapidly to the first encountered local minima, whereas SGD exhibited a more gradual exploration of the loss landscape. Since both optimizers reached comparable final performance on the adopted dataset, we selected *SGD* for the main runs to ensure a fair and stable comparison across LeNet300 and Kanet300 variants. This choice is also consistent with the broader experimental framework used in our KAN/MLP comparisons, where SGD-based protocols are standard.

Search space. Each model (LeNet300 and Kanet300) was trained under the following grid of hyperparameters:

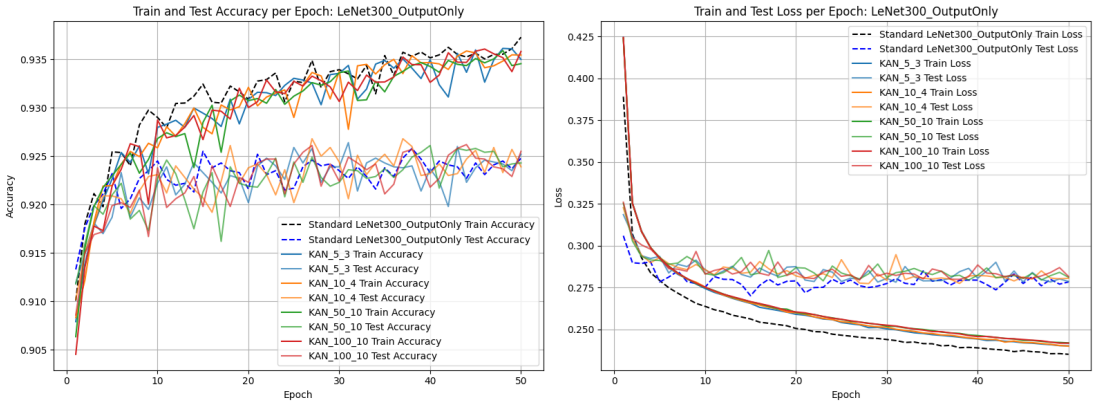
- **Learning rate** $lr \in \{10^{-2}, 10^{-3}, 10^{-4}\}$.
- **Exponential learning-rate decay factor** $\gamma \in \{0.90, 0.95, 0.99\}$.
- **Momentum (for SGD)** $m \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$.
- **Weight regularization**: ℓ_2, ℓ_1, and `no_norm`.

Reported configuration. Among the various settings explored, the following configuration was selected for reporting in the paper. While it did not consistently yield the absolute best results, it achieved performance comparable to the strongest alternatives without introducing additional elements that would complicate the comparison between *LeNet300* and *KANet300*. This choice ensures a fair and balanced baseline for interpreting the observed differences:

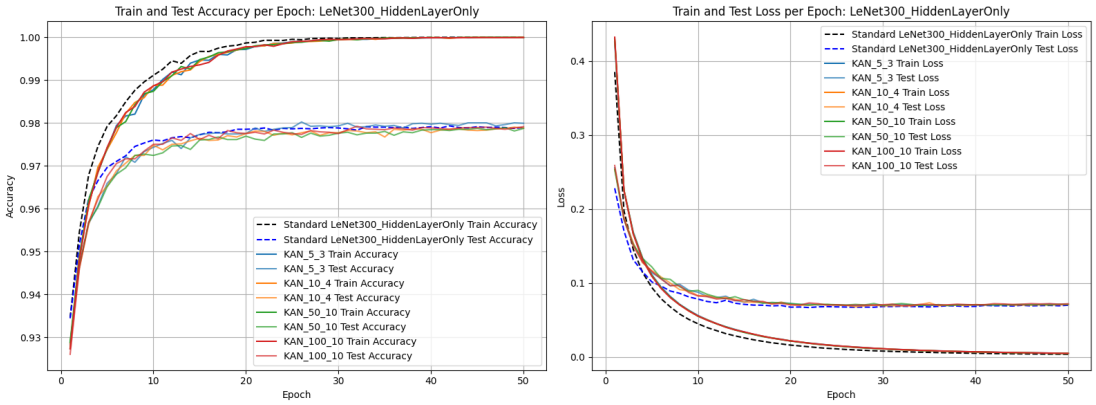
- **Loss function**: `Cross Entropy`.
- **Optimizer**: SGD.
- **Learning rate**: 10^{-2}.
- **Exponential LR decay**: $\gamma = 0.95$.
- **Momentum**: $m = 0$.
- **Type of norm**: `no_norm`.
- **KAN specifics**: ReLU preactivation; grid_size = 5, spline_order = 3.

On the choice of spline hyperparameters.

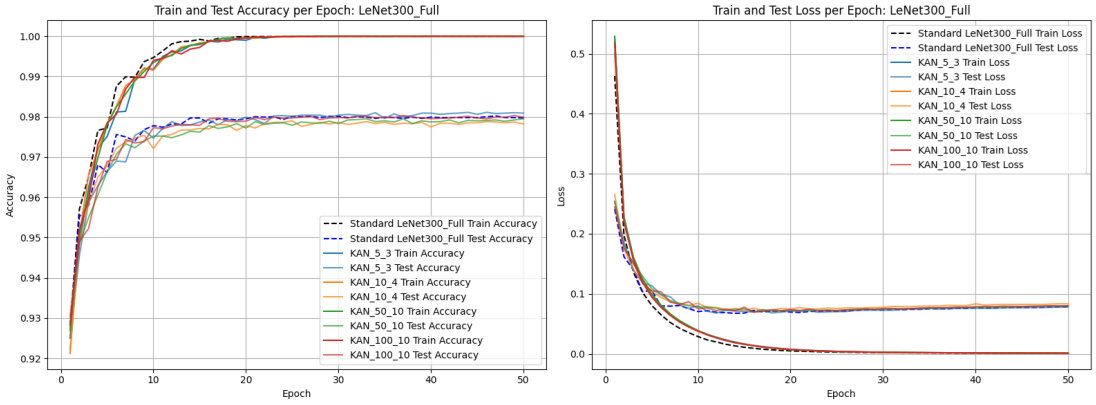
A series of experiments were conducted to assess the impact of spline parameterization on KAN performance. It was observed that when the number of grid points exceeded 5 and the spline order was higher than 3, the learned activation functions across edges were statistically very similar to those obtained with lower settings. Moreover, empirical evidence showed that adopting larger values did not lead to performance improvements on the target task. For this reason, all Kanet300 models were implemented with *gridsize = 5* and *spline_order = 3*, which proved sufficient to capture the necessary nonlinear structure while avoiding unnecessary model complexity.



(a) LeNet300_OutputOnly: accuracy and loss (Train/Test) per epoch for KAN variants compared to the standard model.



(b) LeNet300_HiddenLayerOnly: comparison between standard and KAN versions (Train/Test).



(c) LeNet300_Full: two hidden layers; comparison between standard and KAN over 50 epochs.

Figure 5.5: Experimental results on MNIST for three LeNet-300 configurations with and without KAN. Each panel shows (left) Train/Test accuracy and (right) Train/Test loss per epoch.

5.0.4 Results

From the analysis of Figures 5.6,5.7,and 5.8, several key observations can be made:

- **Loss stability with depth.** As the number of layers increases, the loss trajectory of *LeNet300* appears more stable than that of *KANet300*. The KAN curves exhibit occasional spikes and mild oscillations across epochs, while LeNet is smoother. This behavior is consistent with the observation that the univariate functions used in Kolmogorov–Arnold representations need not be continuous; therefore, when represented with B-splines, a continuity mismatch may hinder optimization and manifest as higher-variance loss curves.

- **Systematic loss gap.** Across all three configurations (OutputOnly, HiddenLayerOnly, Full), the LeNet variants achieve a *slightly* lower loss than the corresponding KAN models throughout training and evaluation. The gap is small but persistent.

- **No advantage when hidden layers are removed.** Contrary to expectations, KAN does not provide a measurable benefit in the "HiddenLayerOnly" and "OutputOnly" setups, despite having learnable activation functions. Both accuracy and loss curves track their LeNet counterparts closely, with LeNet often being marginally better.

- **Overall MNIST performance parity.** On MNIST, KAN neurons do not confer a substantial advantage in classification accuracy: both families reach comparable test/validation plateaus (roughly $\sim$0.98 in the full model and $\sim$0.92–0.93 in the output-only model), with LeNet typically enjoying a small edge in loss.

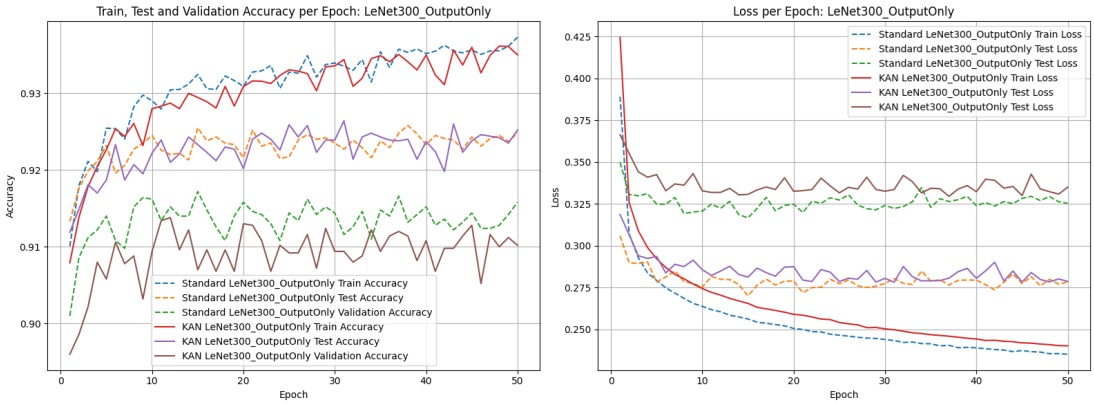


Figure 5.6: LeNet300_OutputOnly vs. KAN: Train/Test/Validation accuracy (left) and loss (right) per epoch.

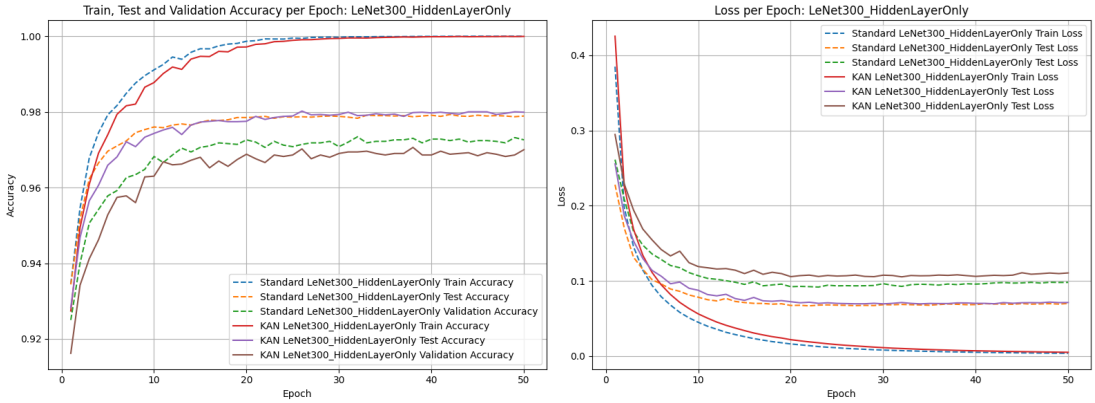


Figure 5.7: LeNet300_HiddenLayerOnly vs. KAN: Train/Test/Validation accuracy (left) and loss (right) per epoch.

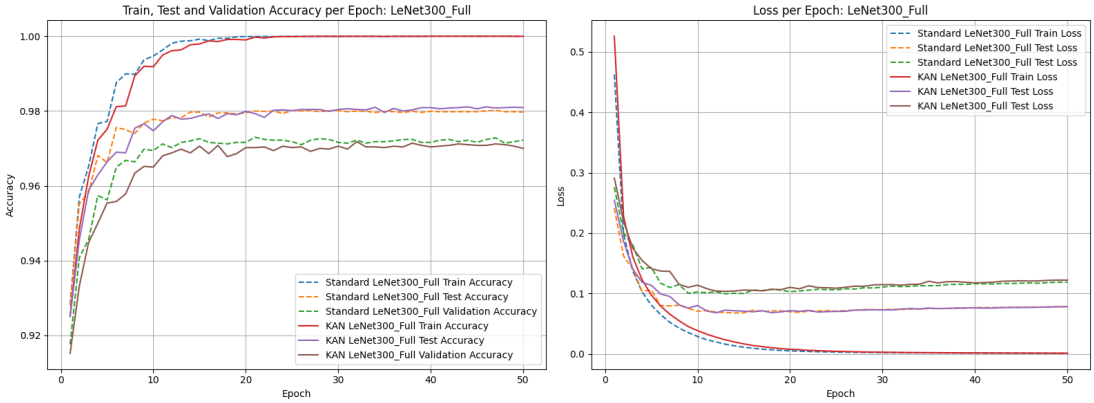


Figure 5.8: LeNet300_Full vs. KAN: Train/Test/Validation accuracy (left) and loss (right) per epoch.

Future hypotheses to test

Motivated by the above 5.6,5.7,and 5.8, we list concrete hypotheses and follow-ups:

1. **Task simplicity.** MNIST may be too easy to reveal the potential benefits of KAN neurons. Replicate on more challenging datasets (Fashion-MNIST, EMNIST, CIFAR-10/100).

2. **Extension to convolutional architectures.** Thus far, experiments have been restricted to fully connected networks. A natural next step is to incorporate KAN neurons into convolutional layers, evaluating whether spline-based activations can provide improvements in hierarchical feature extraction typical of CNNs.

3. **Losses and optimizers.** Different loss functions (e.g., label smoothing, focal loss) and optimizers/schedules may interact better with spline-based activations and change the outcome in favor of KAN.

4. **Initialization and knot placement.** Study sensitivity to activation initialization (e.g., near-linear vs. curved) and to knot placement (uniform vs. data-driven). Adaptive knot updates or annealing the number/order of spline segments across epochs may stabilize training.

5. **Robustness and variability.** Quantify run-to-run variance across random seeds; analyze robustness to label noise and input corruptions, where flexible univariate mappings might offer advantages.

Chapter 6

Experiments with Convolutional KANs

6.1 Introduction

6.1.1 Context and Objective

In this project we analyze the behavior of two convolutional network models using the E-MNIST dataset [6]. This dataset, derived from MNIST, includes both digits and letters (upper- and lowercase) for a total of 62 classes, making it a more complex classification problem than the classic MNIST.
The choice of E-MNIST is particularly relevant: in preliminary studies with the standard MNIST dataset, no competitive advantage of KaNet300 over its LeNet300 counterpart was observed, with LeNet consistently matching or surpassing the KAN-based architecture. For this reason, we adopted the more challenging E-MNIST benchmark to assess whether the additional expressive power of KAN-based filters could provide benefits in a scenario with increased class diversity and complexity.

6.1.2 Motivations

We deliberately employed convolutional architectures rather than purely fully connected ones, since convolutional networks are not only more suitable for image classification tasks but also allow the use of established explainability techniques (xAI) that are typically designed for convolutional layers. This enables a fairer and more insightful comparison of interpretability between LeNet5 and its KAN-based variant. The objective is therefore twofold: to evaluate performance differences in a more complex dataset and to investigate whether the spline-based convolutional filters of ConvKAN offer interpretability or generalization advantages compared to the classical LeNet5 configuration.

6.1.3 Extension to ConvKAN

ConvKAN extend the principles of KAN to convolution operations. Instead of using rigid linear filters, the filters are represented by learned univariate spline functions during training. Parameters such as `grid_size` and `spline_order`

control the complexity of these splines, thus enabling ConvKAN to capture more complex patterns and, potentially, offer better interpretability.

6.2 Architecture setup

6.2.1 LeNet-5

The architecture known as **LeNet-5**, proposed by Yann LeCun and collaborators in 1998, is widely regarded as one of the foundational convolutional neural networks (CNNs). Originally developed for handwritten digit recognition on the **MNIST dataset**, LeNet-5 demonstrated the effectiveness of CNNs in computer vision tasks and laid the groundwork for the modern architectures used today.

In our case, since we employed the **EMNIST dataset** [6], which extends MNIST to include both digits and letters, we modified the **final fully connected layer** in order to adapt the network to the increased number of output classes. Specifically, while the original LeNet-5 had 10 outputs corresponding to the 10 digits, our implementation used 62 outputs to match the EMNIST classification task.

LeNet-5 follows the following sequence:

- **Convolution:** 1 input, 6 output, kernel 5×5, padding=2, followed by a ReLU activation.
- **Pooling:** 2×2 window, reducing the size from 28×28 to 14×14.
- **Convolution:** 6 input, 16 output, kernel 5×5, without padding, followed by a ReLU activation.
- **Pooling:** 2×2 window, reducing the size from 10×10 to 5×5.
- **Fully Connected Layers:** A sequence of layers with 256, 120, 84, and finally **62 neurons**, with intermediate ReLU activations.

The filters used are linear, and this network serves as a consolidated baseline. However, its importance extends far beyond a simple architecture: it introduced key concepts that would later become standard in deep learning.

Peculiarities of the Architecture

LeNet-5 introduced several design principles that remain crucial in modern CNNs:

1. **Local receptive fields (convolution):** drastically reduce the number of parameters compared to fully connected networks, exploiting the spatial structure of the input.
2. **Weight sharing:** convolutional filters are applied across the entire image, enabling translation invariance and parameter efficiency.

3. **Pooling (subsampling):** increases robustness to small shifts and deformations.

4. **Hierarchical feature learning:** early layers capture simple features (edges, corners), while deeper layers capture more abstract concepts.

5. **Non-linearity:** although the original LeNet-5 used sigmoids or hyperbolic tangents, modern versions typically replace them with ReLU for faster convergence and better stability.

Advantages and Limitations

Advantages:

- First architecture to significantly outperform traditional methods on digit recognition tasks.
- Reduced number of parameters through convolutional weight sharing.
- Demonstrated strong generalization capabilities on simple datasets.

Limitations:

- Computationally demanding for the hardware of the 1990s, though negligible today.
- Limited scalability to larger and more complex datasets such as ImageNet.
- Lacks modern improvements such as batch normalization, dropout, and residual connections, which are now standard.

Graphical Representation

The architecture is often illustrated as in Figure 6.1, which provides a visual overview of the sequential layers.

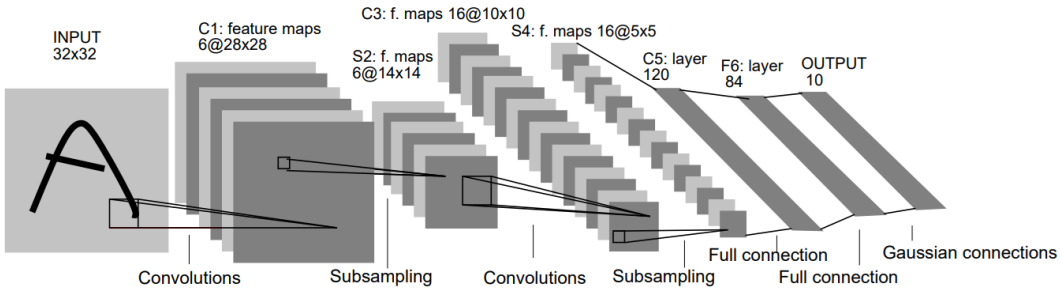


Figure 6.1: Architecture of LeNet-5, Source:[3].

6.2.2 Kanet5

ConvKAN uses layers called `KAN_Convolutional_Layer` based on splines. Below is the KAN topology used in the Kanet5 variant:

- **First conv KAN layer:**
  - `in_channels`=1, `out_channels`=6, kernel 5×5, stride=1, padding=0.
  - `grid_size`=5, `spline_order`=3, with ReLU as the base activation.
  - 2×2 Pooling that reduces the size from 28×28 to 12×12 (a 5×5 convolution without padding yields 24×24, then pooling halves the size).
- **Second conv KAN layer:**
  - `in_channels`=6, `out_channels`=16, kernel 5×5, stride=1, without padding.
  - Same spline settings (`grid_size`=5, `spline_order`=3).
  - 2×2 Pooling that reduces the size from 8×8 to 4×4.
- **Fully Connected Layers:**
  - 256 neurons input (obtained from 16 channels of size 4×4).
  - Sequence of layers: $256 \to 120 \to 84 \to 62$, with intermediate ReLU activations.

In this topology, each filter is a learned spline function, making the filters nonlinear and adaptive.

Topology Comparability

To ensure a fair *like-for-like* comparison, we keep depth, pooling schedule, and classifier width aligned; the main variation lies in the *parameterization of the filters*. In this framework, LeNet5 and Kanet5 share the same macro–organization (hierarchical feature extraction via convolution and pooling, followed by a fully connected head), but they differ substantially in the way weights are represented and applied.

Aspect	LeNet5	ConvKAN
Convolutional filter type	Linear kernels with scalar weights; nonlinearity applied at nodes	Filters as univariate spline functions (defined on edges); nodes perform summation
Placement of nonlinearity	After linear operations (e.g., ReLU or tanh)	Embedded in functional weights (via `grid_size` and `spline_order`)
Pooling	Identical layers and downsampling factors	Identical
Fully connected head	First FC layer with 256 units	Identical (256 units)

6.2.3 Experimental protocol

Dataset

We use the EMNIST dataset comprising digits and letters for a total of 62 classes. Images are grayscale with spatial resolution 28×28. The official training split is used to train the models, while a validation set is obtained by holding out 10% of the training samples; the original test split is used exclusively for final evaluation. All inputs are normalized channel-wise using mean = 0.1307 and std = 0.3081, after converting pixel values to $[0, 1]$.

Hyperparameters and optimizer

Several configurations of hyperparameters were explored, following the same choices adopted in the Lenet300 vs Kanet300 experiments for the layer ablation study. Among these, the configuration reported below achieved the best results for both LeNet5 and KaNet5:

- **Optimizer:** Stochastic Gradient Descent (SGD) without momentum.
- **Learning rate:** 0.01 (fixed).
- **Batch size:** 100.
- **Epochs:** 50.
- **Loss:** Cross-Entropy.
- **Regularization settings:** two regimes are considered:
  - *No Norm:* no L2 weight decay.
  - *L2 Norm:* L2 weight decay enabled.
- **KA-specific settings (KaNet5):** `grid_size` = 5, `spline_order` = 3.

6.2.4 Results

Performance Analysis

The learning dynamics in Figs. 6.2, 6.3, 6.4, and 6.5 (LeNet5 *vs.* KaNet5 under L2 and NoNorm regimes) indicate that, while both architectures achieve very similar overall performance, CNNs consistently hold a slight edge of about 0.1 percentage points in accuracy. This mirrors the results reported in [7] on the Fashion-MNIST dataset, where convolutional baselines also surpassed convolutional KANs by roughly the same margin. Although the gap is small, it appears systematic across tasks, with total precision values remaining in a comparable range. In particular, we observe:

- **Fast early improvement.** During the first few epochs, accuracy rises quickly and loss drops sharply; thereafter both curves flatten and progress becomes incremental across all settings.
- **LeNet5 vs KaNet5.** At matched regularization, the dashed LeNet5 curves lie consistently above the solid KaNet5 curves for *train*, *test*, and *validation* accuracy (Figs. 6.4 and 6.5). The margin is modest and fairly stable over epochs. The loss plots mirror this: LeNet5 maintains slightly lower loss than KaNet5 across all splits (Figs. 6.2 and 6.3).
- **Effect of L2 vs NoNorm.** Comparing L2 to NoNorm, the *loss* curves with L2 sit a bit lower for both architectures and all splits (Fig. 6.2 vs. 6.3). In contrast, the *accuracy* curves with L2 and NoNorm are very close; any differences are minor and not systematically in favor of one regime (Fig. 6.4 vs. 6.5).
- **Generalization gap.** Train and test/validation curves remain close throughout training (typically within about one percentage point near the end), indicating limited overfitting under both regularization settings.
- **Plateau.** All configurations show diminishing returns after roughly epochs 25–30: validation accuracy changes little while training loss continues to decrease slowly, suggesting that longer training offers limited benefit without additional regularization or early stopping.

Summary. Across both regularization regimes, **LeNet5** retains a small but consistent edge over **KaNet5** in both accuracy and loss. **L2** regularization reliably lowers the loss relative to **NoNorm**, while delivering only marginal (if any) accuracy changes. The gaps between training and validation/test remain small, and learning saturates by the late epochs.

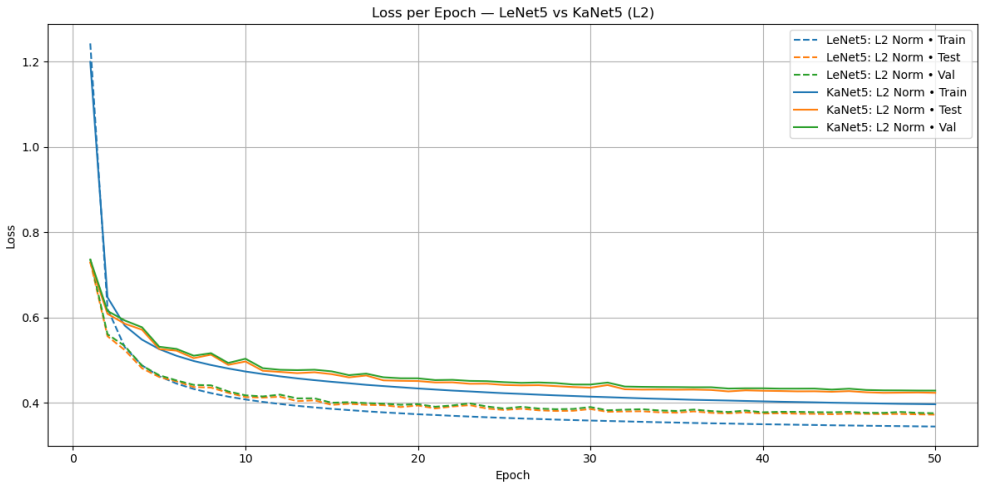


Figure 6.2: Loss trend over epochs for LeNet5 vs KaNet5 with L2 normalization.

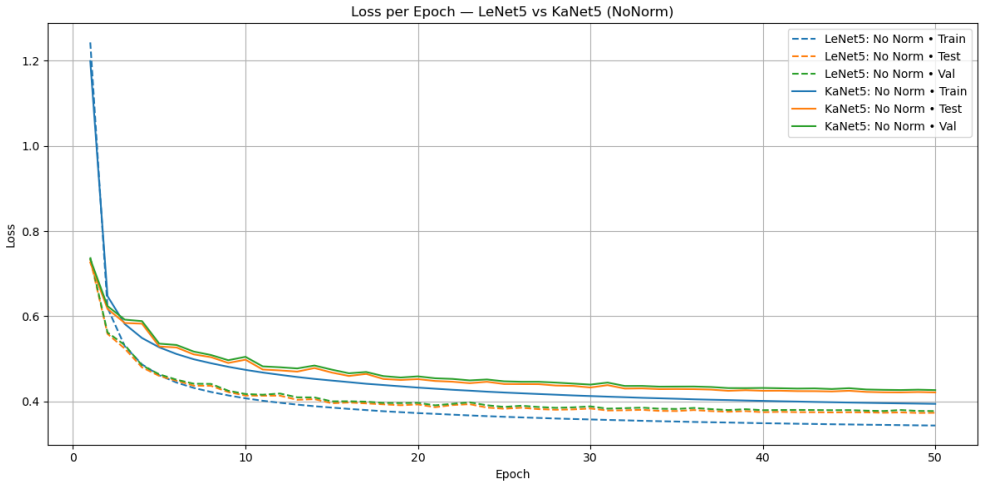


Figure 6.3: Loss trend over epochs for LeNet5 vs KaNet5 without normalization (NoNorm).

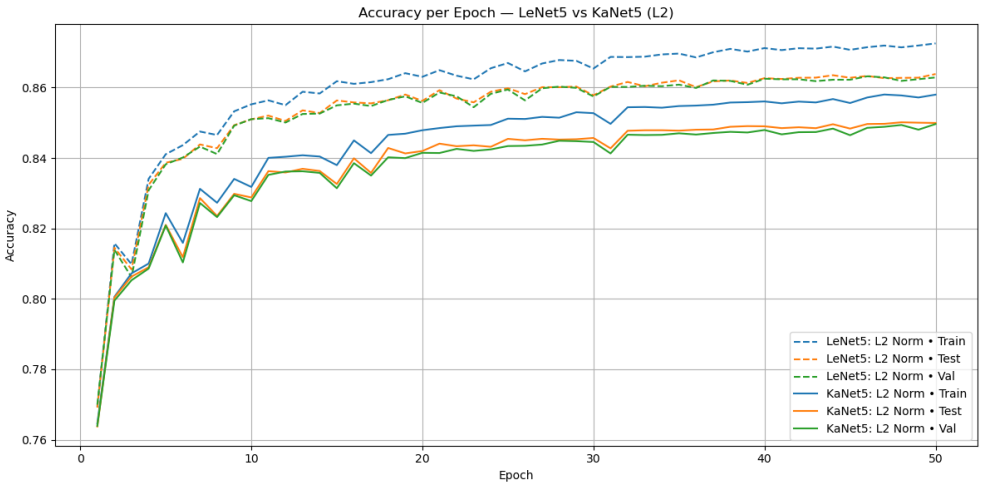


Figure 6.4: Accuracy trend over epochs for LeNet5 vs KaNet5 with L2 normalization.

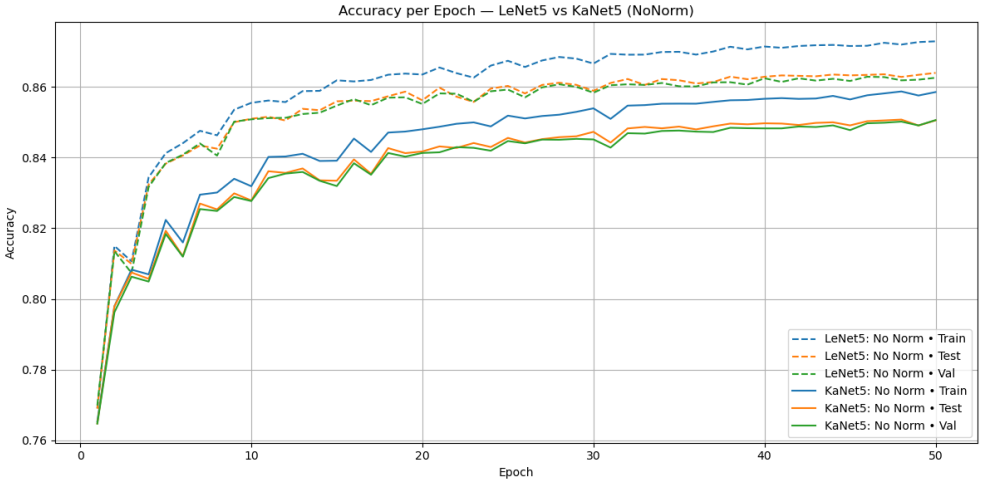


Figure 6.5: Accuracy trend over epochs for LeNet5 vs KaNet5 without normalization (NoNorm).

Chapter 7

Interpretability of KAN networks: Structural Alignment of Attention and Statistical Evaluation

This chapter investigates the internal evidence used by the two reference architectures — Standard LeNet5 and KaNet5 (based on ConvKAN)—to support their predictions. The objective is to provide a rigorous and transparent account of *where* each model attends and *how* such attention changes under different normalisation schemes and evaluation criteria. To this end, the analysis combines activation-based views (*feature maps*) with class-discriminative explanations (*Grad-CAM*) and evaluates their behaviour through a set of structural metrics and statistical tests designed to meet publication standards.

Methodologically, feature maps are extracted from the last convolutional stage and subjected to several normalisation strategies in order to distinguish effects driven by absolute magnitude from those reflecting relative prioritisation. Grad-CAM is then adapted to KaNet5 and applied consistently across models to generate class-specific heat maps that highlight discriminative regions. These maps are quantified against interpretable geometric masks—such as edges, corners, loops, concavities and vertical structures—and aggregated using robust, non-parametric procedures to assess both effect sizes and statistical significance.

7.1 Grad-CAM principles and applications

Gradient-weighted Class Activation Mapping (Grad-CAM) [9] is a class-discriminative localisation technique that attributes a model's decision to spatial regions in the input by reweighting the last convolutional feature maps with the gradient of a target output. Consider the feature maps $A^k \in \mathbb{R}^{u \times v}$ of the last convolutional layer and the pre-softmax score y^c for class c. Channel importances are obtained by global-average pooling the gradients of the class score with respect to the feature maps,

$$\alpha_k^c \;=\; \frac{1}{Z}\sum_{i=1}^{u}\sum_{j=1}^{v}\frac{\partial y^c}{\partial A_{ij}^k}, \qquad Z = uv, \tag{7.1}$$

and the class-specific saliency map is the rectified linear combination

$$L^c_{\text{Grad-CAM}} \;=\; \text{ReLU}\left(\sum_k \alpha_k^c \, A^k\right), \tag{7.2}$$

which highlights spatial locations exerting a positive influence on y^c. In practice, a single forward pass followed by a backward pass that stops at the chosen convolutional layer suffices to compute $L^c_{\text{Grad-CAM}}$; the map is then upsampled to input resolution for inspection or downstream scoring.

What Grad-CAM provides. Grad-CAM yields *class-discriminative*, *human-readable* heat maps without modifying or retraining the model. Because the construction depends only on the gradients flowing into the final convolutional representation, the method applies broadly to CNN-based architectures used for classification, captioning, VQA and other vision tasks. The heat maps supply a direct, spatial answer to the question "*where in the image did the network find evidence for class c?*", enabling visual auditing of decisions and comparison across samples, classes and training regimes.

Why it is useful. From a scientific perspective, Grad-CAM supports model debugging and failure analysis: false positives can be traced to spurious textures or backgrounds; false negatives often reveal missing or weak evidence in the expected region. From an engineering perspective, Grad-CAM assists data curation (identifying systematic dataset bias), architecture selection (verifying that design changes shift attention to intended structures), and deployment-time monitoring (auditing whether production predictions are grounded on legitimate regions). In user-facing settings, its explanations can calibrate trust by showing that seemingly unreasonable predictions are in fact supported by reasonable evidence; conversely, they can expose shortcut learning when heat concentrates on irrelevant artefacts.

Design choices and good practice. Three choices materially affect map quality. *Layer selection:* using the last convolutional block balances semantic specificity with spatial resolution; earlier layers improve resolution but risk highlighting generic edges rather than class evidence. *Target score:* pre-softmax logits are preferred to probabilities to avoid gradient damping near saturated softmax outputs. *Rectification and scaling:* the ReLU in (7.2) filters inhibitory evidence, producing maps that indicate features *supporting* class c. For quantitative use, it is advisable to report both the raw $L^c_{\text{Grad-CAM}}$ and normalised variants (e.g., min–max or z-score) and to state clearly any thresholding, smoothing or upsampling scheme applied. When multiple classes are plausible, one may compute $L^c_{\text{Grad-CAM}}$ for each candidate and inspect differences; for top-k predictions, averaging or taking maxima across classes must be interpreted cautiously, as it changes the attribution question.

Faithfulness and limitations. Grad-CAM reflects the *local* linear sensitivity of the output to the last convolutional features; it is faithful to that representation but inherits its stride and receptive-field coarseness. Maps may blur object boundaries or appear fragmented when the chosen layer has low spatial resolution. Because the method relies on gradients, saturated or noisy gradients can degrade explanations; smoothing (e.g., minor Gaussian blur) and careful numerical settings mitigate this. Negative evidence is suppressed by the ReLU, which improves visual readability but omits regions that *argue against* the class; analysing both the rectified and the raw linear combination can therefore be informative. Finally, Grad-CAM attributes *evidence* rather than *causal necessity*; complementary tests (deletion/insertion, counterfactual masking) are recommended when causal claims are important.

Relationship to other techniques. Class Activation Mapping (CAM) is recovered as a special case when the architecture imposes global-average pooling before the classifier; Grad-CAM generalises this idea to standard CNNs without architectural constraints. Pixel-space gradient methods provide high-frequency detail but are not class-discriminative; combining them multiplicatively with an upsampled $L^c_{\text{Grad-CAM}}$ (Guided Grad-CAM) merges fine detail with class selectivity. Numerous extensions (e.g., confidence-weighted or higher-order variants) exist, yet the original formulation in (7.1)–(7.2) remains the most broadly applicable baseline for publication-grade comparisons.

7.2 Feature map extraction

Objective and setting. Feature maps are the spatial activation tensors produced by the last convolutional stage. They provide a direct, architecture-agnostic view of a network's *sensitivity* to the input. In our implementation, we instrument the KaNet5 model (a LeNet5-style backbone with spline-based convolutional layers) to expose these maps during inference and to summarise their basic statistics for subsequent analyses.

Model hook and tensor captured. The network `LeNet5_KAN` replaces classical convolutions with `KAN_Convolutional_Layer`s (cubic splines over a fixed grid with ReLU base activation). The forward pass applies two convolution–pooling blocks followed by three fully connected layers:

$$\text{conv}_1 \;\to\; \text{avgpool}_1 \;\to\; \text{conv}_2 \;\to\; \text{avgpool}_2 \;\to\; \text{FCs}.$$

We intercept the output of the *second* convolution, immediately *before* the second pooling, by storing the tensor in the attribute `last_conv_feat`:

$$F \equiv \texttt{model.last\_conv\_feat} \in \mathbb{R}^{B\times C\times H\times W} \quad \text{with} \quad (C,H,W) = (16,8,8).$$

Dimensions follow from EMNIST inputs (28×28, one channel), valid 5×5 convolutions (no padding) and 2×2 average pooling: $28 \to 24 \to 12 \to 8$ at the

tap point. The captured maps are thus *pre-pooling* responses at the deepest convolutional level. They are not upsampled here; upsampling to the input grid is applied only when spatial alignment with masks is required in later sections.

Data and reproducibility. Test and training images are read directly from the EMNIST `idx` files into memory (`uint8` $\rightarrow$ `float`), wrapped in `Dataset`/`DataLoader` objects, and iterated in mini-batches. To ensure repeatability, we fix seeds for NumPy, Python and PyTorch and place the model in `eval` mode. A pre-trained checkpoint is loaded from the most recent file in `model/`, preferring filenames of the form `model_epoch_{E}.pth`; otherwise creation time is used as a tiebreaker.

Per-image statistics. For each batch we obtain logits $y = \texttt{model}(x)$ and the predicted class $\hat{c} = \arg\max y$. At the same time we retrieve the saved feature maps F. Let $F_n \in \mathbb{R}^{C\times H\times W}$ denote the tensor for image n (with $C = 16$, $H = W = 8$). We flatten F_n over channels and spatial coordinates and compute three scalar summaries:

$$\mu_n \;=\; \frac{1}{CHW}\sum_{k,i,j} F_{n,kij}, \qquad s_n^2 \;=\; \frac{1}{CHW-1}\sum_{k,i,j}(F_{n,kij}-\mu_n)^2, \qquad m_n \;=\; \max_{k,i,j} F_{n,kij}. \tag{7.3}$$

These quantities represent, respectively, the *mean intensity* (overall activation level), the *intensity variance* (dispersion of responses) and the *maximum intensity* (peak response) of the deepest convolutional representation.

Stratification by correctness and split. Each image is assigned to one of four disjoint categories according to the data split and prediction correctness:

$$\mathcal{S} \in \{\text{train/correct},\ \text{train/incorrect},\ \text{test/correct},\ \text{test/incorrect}\}.$$

For every $\mathcal{S}$ we accumulate running sums of μ_n, s_n^2 and m_n together with the count $N_{\mathcal{S}}$. After a full pass over the loaders, category-wise averages are reported as

$$\overline{\mu}_{\mathcal{S}} = \frac{1}{N_{\mathcal{S}}}\sum_{n\in\mathcal{S}} \mu_n, \qquad \overline{s^2}_{\mathcal{S}} = \frac{1}{N_{\mathcal{S}}}\sum_{n\in\mathcal{S}} s_n^2, \qquad \overline{m}_{\mathcal{S}} = \frac{1}{N_{\mathcal{S}}}\sum_{n\in\mathcal{S}} m_n. \tag{7.4}$$

The results are saved as CSV files (one per category) with headers `mean_intensity`, `variance_intensity`, `max_intensity`. This compact summary supports downstream hypothesis testing and cross-model comparisons without reprocessing the full tensor volumes.

Implementation notes. (1) Inputs are intentionally kept in their original $[0, 255]$ scale (no per-image normalisation) to preserve absolute activation magnitudes; this choice simplifies interpretation of μ_n and m_n but makes them sensitive to input scaling. (2) Capturing feature maps *before* the second pooling preserves higher spatial resolution (8×8) at the deepest level available in the architecture. (3) The accumulation is vectorised over batch elements for efficiency; no gradients are required and the loop runs under `torch.no_grad()`.

7.2.1 Grad-CAM on KaNet5

Why it still works out of the box. KaNet5 replaces fixed convolutional kernels with *spline filters* (learned weights parameterized by spline bases). Despite this, Grad-CAM needs no custom code: PyTorch autograd propagates gradients through the spline interpolation, so `pytorch_grad_cam` can read activations and their gradients exactly as with standard convolutions.

Where we hook (pre-activation). To make the comparison fair and avoid any nonlinearity-specific effects, the Grad-CAM target in KaNet5 is the *pre-activation feature maps* of the second convolutional block (the same stage used for LeNet5). In practice we pass the module that emits those pre-activation maps inside `conv2` to `pytorch_grad_cam`'s `target_layers`.

Everything else is identical. Target class selection, batching, checkpoints, smoothing options, and output artifacts are exactly as in the LeNet5 pipeline already described; no further differences are introduced for KaNet5.

7.3 Class-specific heat maps

7.3.1 Visualizations of Grad-CAM

The qualitative inspection of the Grad-CAM visualizations provides useful insights into how LeNet5 and ConvKAN allocate their attention when processing the same input.

- **Correct Examples:** Both networks show focused heatmaps. LeNet5 highlights the essential features of the character, while ConvKAN concentrates on the main strokes, although with more diffuse activations.
- **Incorrect Examples:** LeNet5's heatmaps appear less sharp yet coherent, whereas those of ConvKAN seem more "sparse," reflecting a higher internal complexity.
- **Examples where one network fails and the other succeeds:** Differences in attention distribution are evident; ConvKAN may detect patterns that LeNet5 tends to ignore (or vice versa).

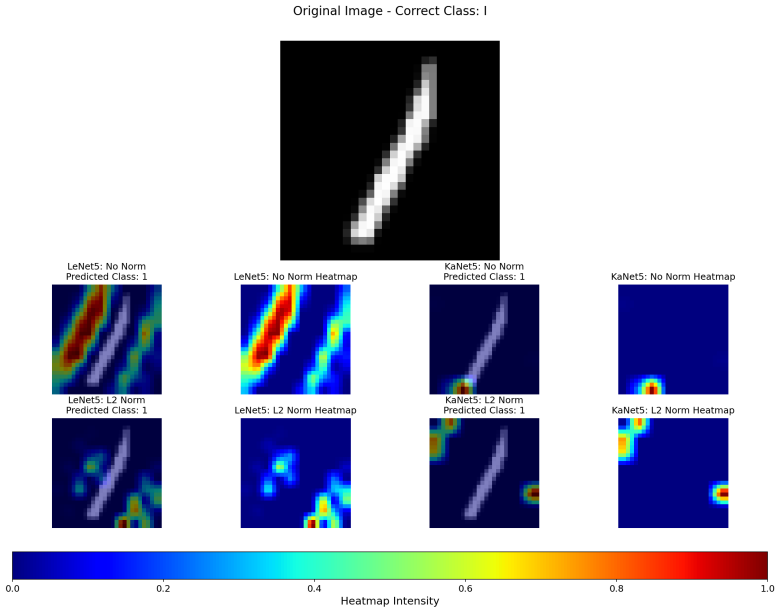


Figure 7.1: Grad-CAM visualization for the character "I".

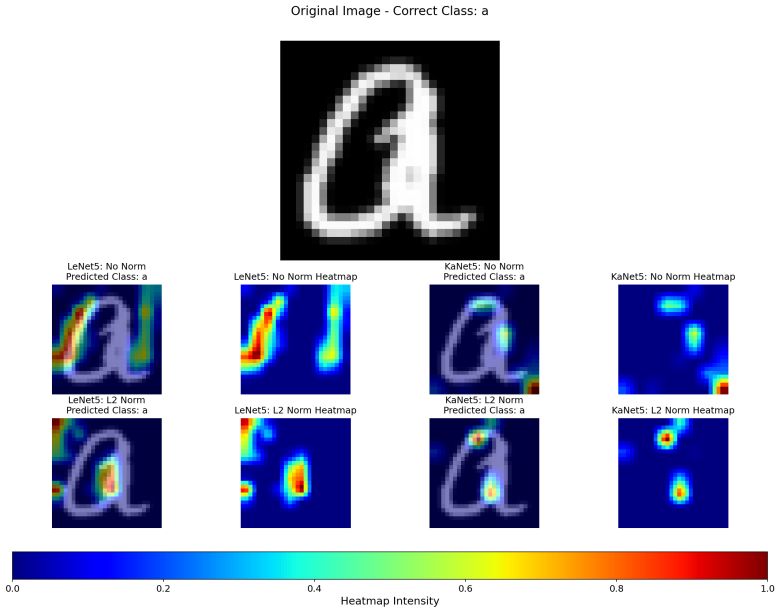


Figure 7.2: Grad-CAM visualization for the character "a".

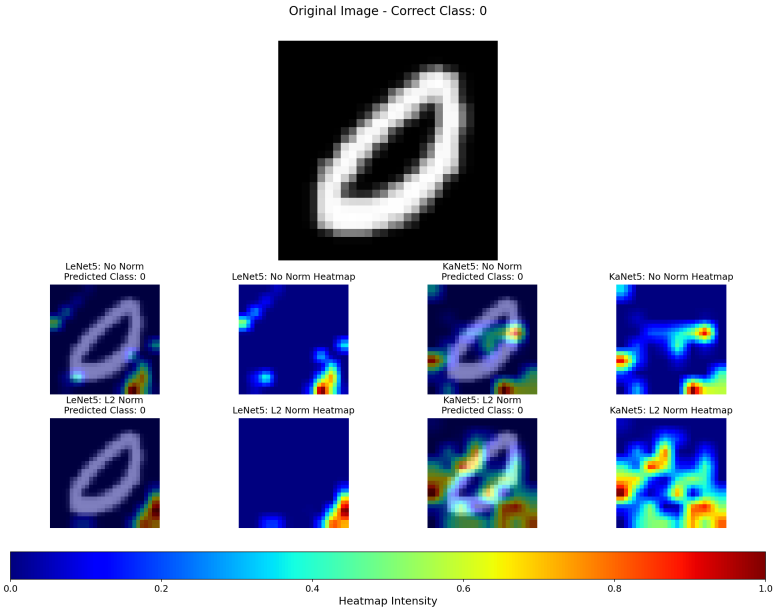


Figure 7.3: Grad-CAM visualization for the digit "0".

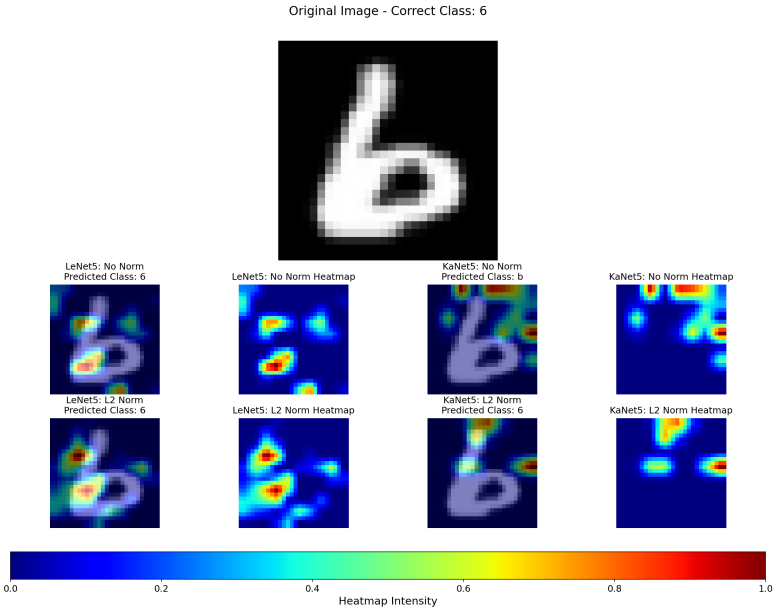


Figure 7.4: Grad-CAM visualization for the digit "6".

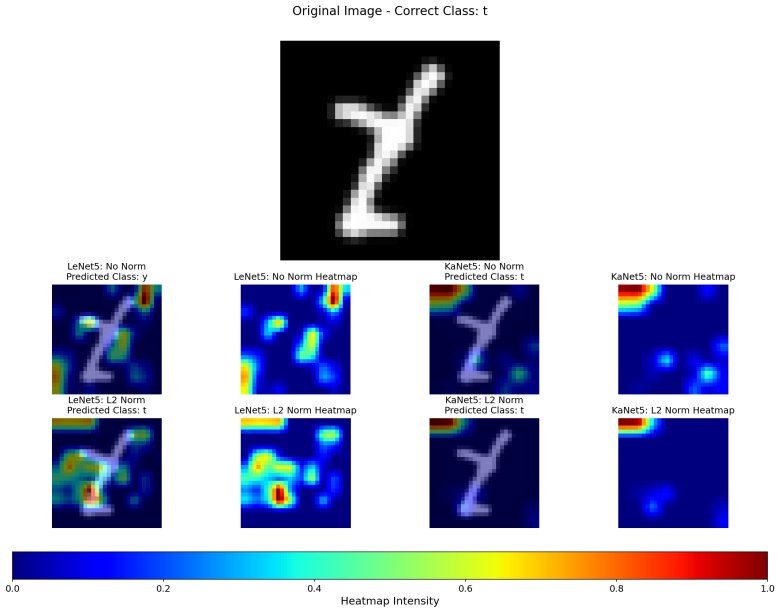


Figure 7.5: Grad-CAM visualization for the character "t".

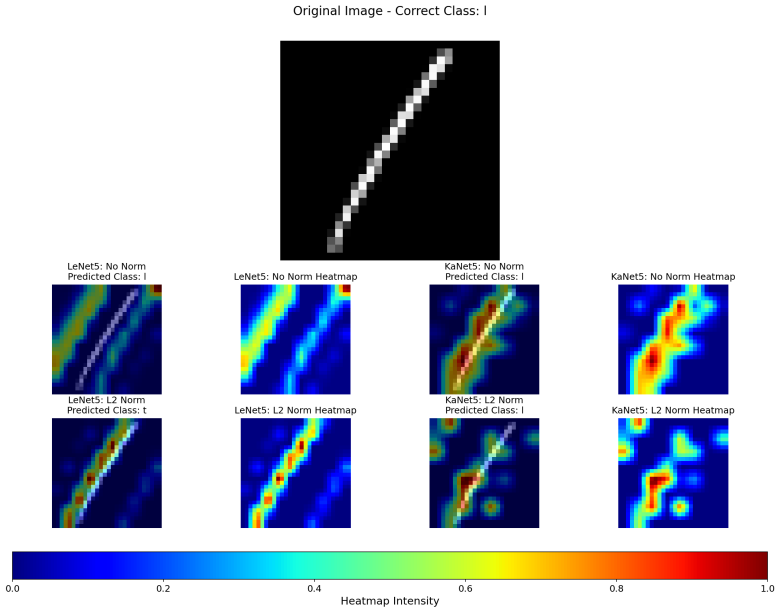


Figure 7.6: Grad-CAM visualization for another instance of the character "I".

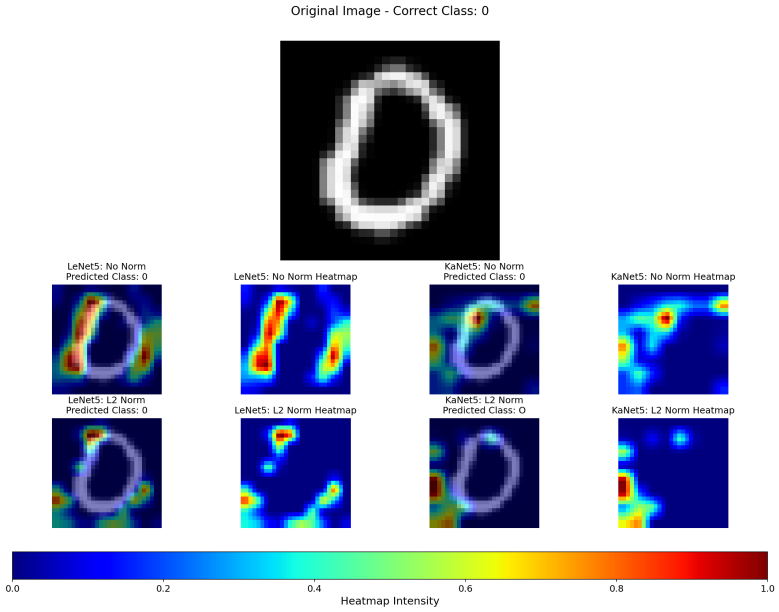


Figure 7.7: Grad-CAM visualization for another instance of the digit "0".

7.3.2 Gradcam Statistics

The following tables contain the mean, variance, and maximum values of the Grad-CAM intensity. In correctly classified cases, the intensity is more concentrated, whereas in misclassified cases it decreases. ConvKAN generally shows a higher variance in activations.

Table 7.1: GradCAM Heatmap statistics for LeNet5 in No Norm mode

Description	**Mean Intensity**	**Variance**	**Max Intensity**
Train correct	5.44	167.73	91.79
Train incorrect	5.33	156.76	85.21
Test correct	5.44	167.76	91.81
Test incorrect	5.35	158.63	86.20

Table 7.2: GradCAM Heatmap statistics for KaNet5 in No Norm mode

Description	**Mean Intensity**	**Variance**	**Max Intensity**
Train correct	0.54	427.62	70.93
Train incorrect	0.56	367.81	62.29
Test correct	0.54	427.43	70.95
Test incorrect	0.56	374.89	63.16

Table 7.3: GradCAM Heatmap statistics for LeNet5 in L2 Norm mode

Description	**Mean Intensity**	**Variance**	**Max Intensity**
Train correct	6.34	219.08	103.02
Train incorrect	6.16	201.74	95.14
Test correct	6.34	219.13	103.08
Test incorrect	6.17	204.01	96.15

Table 7.4: GradCAM Heatmap statistics for KaNet5 in L2 Norm mode

Description	**Mean Intensity**	**Variance**	**Max Intensity**
Train correct	0.91	576.16	81.58
Train incorrect	0.82	493.62	71.46
Test correct	0.92	575.79	81.59
Test incorrect	0.81	503.85	72.54

7.3.3 Generation and Meaning of the Mean Image

The mean image represents a visual summary of the recurring characteristics in each class of the E-MNIST dataset. In this study, the mean image was obtained by computing the arithmetic average of each pixel across all images belonging to a specific class. The result is a representation that captures the typical shape of the character, highlighting regions where the writing features consistently concentrate.

This approach is particularly useful in OCR applications and Explainable AI (xAI), as it provides an objective reference point for model attention analysis. By calculating the mean separately for each class, a set of representative images for both letters and digits was created, which were then used as a base for overlaying the heatmaps generated by the convolutional networks. This procedure facilitated a direct comparison between LeNet5 and ConvKAN, offering a clear view of the most activated areas.

Figure 7.8: Mean prototype for each class in the E-MNIST dataset.

7.3.4 Role of the Mean Example Analysis

The mean image played an essential role in model interpretability analysis. It was used as a reference for overlaying activation maps—both those derived from the average feature maps and those from GradCAM—thus facilitating the interpretation of the areas of attention. In particular, by overlaying the mean image, it was possible to evaluate whether the activated regions corresponded to the expected features of the character or if unexpected sub-regions emerged.

Differences in Activation between GradCAM and Feature Maps

While the mean feature maps provide an overview of the raw activations from the convolutional layer, GradCAM highlights the regions that have a decisive impact on the final output. In other words, the feature maps indicate *where* the model activates, whereas GradCAM quantifies *how much* each activation contributes to the classification decision.

In our study, LeNet5 tends to produce GradCAM maps that overlap homogeneously across the entire structure of the character, reflecting the global behavior of its linear filters. In contrast, ConvKAN shows more segmented and localized GradCAM maps, consistent with its spline-based architecture. This difference is particularly evident in letters characterized by internal loops or vertical strokes: while LeNet5 provides a continuous coverage, ConvKAN highlights specific areas, sometimes separating them into multiple zones.

7.3.5 Dynamic Visualization System

The web interface—developed using HTML, CSS, and JavaScript—enables an interactive exploration of the evolution of activation maps over time. A live `interactive demo` lets users enter an E−MNIST character string into a text input field, dynamically selecting which images to juxtapose for comparison. The displayed images update in real time, showing how heatmaps evolve across training epochs.

To enhance user experience and facilitate analysis, the interface includes several key features:

- **Auto-loop:** Each video automatically restarts after a short pause at the end of the animation, ensuring continuous playback.
- **Playback Speed Control:** A slider allows users to modify the playback speed nonlinearly, making it possible to observe both rapid and slow heatmap transitions.
- **Navigation and Zoom:** The interface supports pan and zoom functionalities, enabling users to closely examine specific regions of the activation maps.
- **Auto-sizing:** The visualization automatically adjusts to different screen sizes and image dimensions, optimizing the layout for clarity and ease of use.

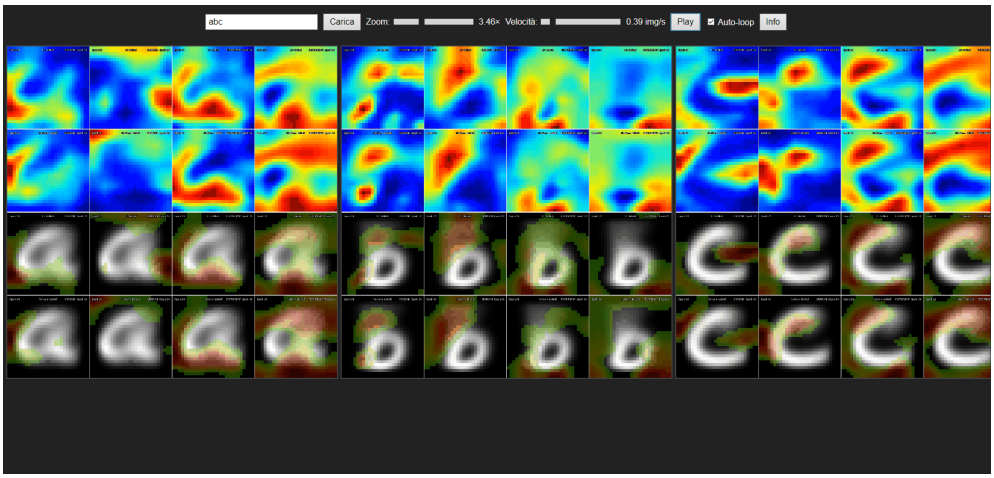


Figure 7.9: Web interface for dynamic visualization of activation maps. Users can select characters to compare, observe heatmap evolution across epochs, and navigate images using pan and zoom functionalities.

7.3.6 Structural attention metrics

Building on the qualitative inspection of mean-class examples and their associated Grad-CAM and feature-map representations, we formulated a set of hypotheses concerning systematic differences between Standard-LeNet5 and KaNet5. To evaluate these hypotheses rigorously, we designed structural attention metrics that quantify the degree of overlap between network activations and interpretable geometric primitives of the EMNIST characters. Unlike the exploratory phase based on class means, the metrics were applied to a controlled subset of 200 randomly selected samples per class from the validation set, thereby ensuring that the analysis captures individual-level variability rather than artifacts of averaging. In this way, the proposed metrics provide a reproducible and statistically grounded framework for comparing the attention profiles of the two architectures.

Normalization Variants

Each raw heat map H is transformed by one of the four type of normalizations. From an XAI perspective each variant highlights different aspects of the models' internal attention:

- **raw**: identity, Preserves both sign and magnitude of the activations, making it possible to see directly which pixels drive excitatory (positive) or inhibitory (negative) responses.
- **norm** ($[0, 1]$ min–max):

$$H_{\text{norm}} = \frac{H - \min H}{\max H - \min H}\,.$$

  Emphasizes relative activation strengths on a common scale, allowing direct visual comparison of where the network focuses, regardless of absolute intensity.

- **abs_ordinal**: rank order of the absolute values,

$$H_{\text{ord}}(i, j) = \text{rank}\Big(|H_{ij}|\Big) \quad \text{(over all pixels)}.$$

  Highlights the most salient pixels by ordering them, while ignoring sign, to reveal which regions are deemed most informative.

- **zscore** (abs-z-min–max):

  1. $A = |H|, \quad Z = \dfrac{A - \mu(A)}{\sigma(A)}$;
  2. $H_{\text{z}} = \dfrac{Z - \min Z}{\max Z - \min Z}$.

  Filters out outliers and centers activations around their mean, then rescales to $[0, 1]$, exposing pixels that deviate most from average activation.

Score Computation

All six pixel-wise scores combine a normalized heat map $H^\star$ with a structural mask or filter extracted from the original EMNIST image. Given a mask M, the generic score is

$$S = \frac{1}{N} \sum_{i,j} H^\star_{ij} \, M_{ij}, \qquad N = \sum_{i,j} \mathbf{1}\{M_{ij} > 0\}.$$

The segmentation variant instead counts activation clusters:

$$S_{\text{segloop}} = \Big| \{\text{unique clusters in quantile-segmented } (H^\star \odot M_{\text{loop}})\} \Big|.$$

`HeatxEdge` The mask M_{edge} is obtained via Sobel operators along x and y, i.e. $M_{\text{edge}} = |\nabla_x I| + |\nabla_y I|$. This measures how much the model relies on object contours and stroke borders.

`HeatxCorner` The mask M_{corner} comes from `cornerMinEigenVal`, which estimates the minimum eigenvalue of the local gradient covariance matrix. It highlights geometric corners and junctions in the character.

`HeatxLoop` The mask M_{loop} is derived from contour hierarchy, filling inner contours detected within the character's binary map. It captures attention inside enclosed regions and cavities (loops).

`HeatxORL_Mask` The mask M_{orl} is built from convexity defects after morphological opening. Concave bays on the glyph boundary are emphasized. This measures the use of concave shape features.

`HeatxVCR_Mask` The mask M_{vcr} is based on vertical-run concentration: columns with the longest vertical continuity (top 10%) are selected. It highlights sensitivity to vertical strokes.

`Segmentation_heat_loop` This score segments $H^\star \odot M_{\text{loop}}$ into 10 quantile bins and counts the number of non-empty clusters:

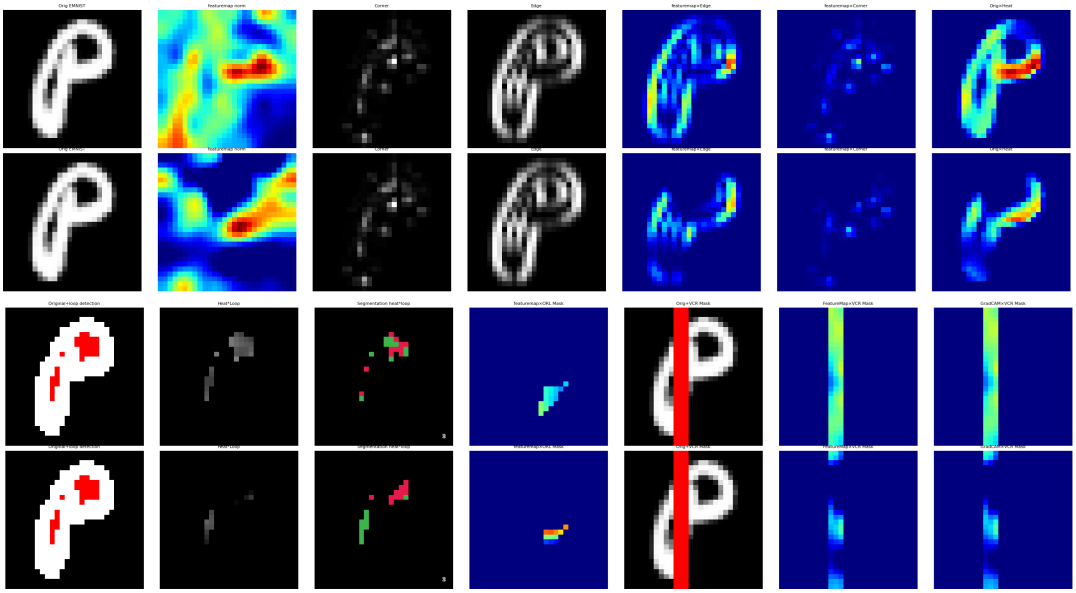
$$S = \#\{1, \ldots, 10 \text{ clusters with nonzero mask}\}.$$

It quantifies the fragmentation of activation patterns inside loops.

Practical pipeline. In practice, the procedure is as follows:

1. Load each upsampled heat map H.
2. Apply one chosen transform: `raw`, `norm`, `abs_ordinal`, or `zscore`.
3. Extract the masks $\{M_{\text{edge}}, M_{\text{corner}}, M_{\text{loop}}, M_{\text{orl}}, M_{\text{vcr}}\}$ from the original EMNIST image.
4. Compute each score S by element-wise multiplication and averaging (or clustering for segmentation).

5. Save the results for later statistical comparison (*Wilcoxon signed-rank*, two-sided; null hypothesis: median difference = 0) between Standard-LeNet5 and KaNet5.



Comparison grid (4×7) of Std_LeNet5 vs. KaNet5 feature maps / Grad-CAM overlays

Legend of the 4×7 Comparison Grid

- **Row 1 (Std_LeNet5):** Orig, FM norm, Corner, Edge, FM×Edge, FM×Corner, Orig+Loop
- **Row 2 (KaNet5):** Orig, FM norm, Corner, Edge, FM×Edge, FM×Corner, Orig+Loop
- **Row 3 (Std_LeNet5):** Orig+Loop det., FM×Loop, Seg._heat_loop, FM×ORL, Orig+VCR, FM×VCR, GradCAM×VCR
- **Row 4 (KaNet5):** Orig+Loop det., FM×Loop, Seg._heat_loop, FM×ORL, Orig+VCR, FM×VCR, GradCAM×VCR

7.3.7 Statistical aggregation

Effect@50 (percent difference). For each metric M and normalization, we compute the scores at epoch 50 for LeNet5 (A_{50}) and KaNet5 (B_{50}). The reported Effect@50 is

$$\text{Effect@50} = 100\,\frac{A_{50} - B_{50}}{|B_{50}|},$$

i.e. the percent difference of LeNet5 relative to KaNet5 at epoch 50 (median over the matched pairs in the batch-level aggregation). Positive values favor LeNet5; negative values favor KaNet5; 0 indicates parity.

Significance and hypothesis (Wilcoxon). Statistical significance is assessed using the Wilcoxon signed-rank test (two-sided). The null hypothesis, consistent with the generated CSV files, is defined as:

$$H_0 : \text{ median difference} = 0.$$

In the tables, the column *Informative? (H_0: median difference = 0)* reports "Yes" or "No," with the corresponding p-value given in parentheses (two decimal digits). A result is considered informative if $p < 0.05$.

The column *Changed* indicates whether the sign of the inequality between LeNet5 and KaNet5 flipped at any point during the 50 epochs. A value "T" (True) means that the relative ordering of the two models changed at least once (i.e., the metric was higher for LeNet5 in some epochs and higher for KaNet5 in others). A value "F" (False) means that the sign of the difference remained constant throughout training, so one model consistently outperformed the other on that metric.

Note on raw scores. Because each pixel-wise score is averaged over the map, `raw` activations can yield extremely large percentage effects whenever one model's mean activation is driven toward zero. In practice, KaNet5 produces many more negative pixel values in feature maps than LeNet5, so its mean raw score at epoch 50 may be close to zero while LeNet5's mean remains strictly positive. As

$$\text{Effect@50} = 100\,\frac{A_{50} - B_{50}}{|B_{50}|},$$

a very small $|B_{50}|$ inflates the reported percentage under `raw` normalization.

Table 7.5: Feature-map Metrics: Standard-LeNet5 vs. KaNet5 (Effect@50, Wilcoxon)

Metric	Norm. type	Effect@50 (%)	Comparison@50	Informative?	Changed
heatxedge	raw	511.62	LeNet5 > KaNet5 by 511.62%	Yes (0.00)	F
heatxedge	norm	-0.45	KaNet5 > LeNet5 by 0.45%	No (0.10)	T
heatxedge	abs_ordinal	8.63	LeNet5 > KaNet5 by 8.63%	Yes (0.00)	F
heatxedge	zscore	62.42	LeNet5 > KaNet5 by 62.42%	Yes (0.00)	F
heatxcorner	raw	479.77	LeNet5 > KaNet5 by 479.77%	Yes (0.00)	F
heatxcorner	norm	-1.62	KaNet5 > LeNet5 by 1.62%	Yes (0.00)	T
heatxcorner	abs_ordinal	2.82	LeNet5 > KaNet5 by 2.82%	Yes (0.00)	F
heatxcorner	zscore	55.80	LeNet5 > KaNet5 by 55.80%	Yes (0.00)	F
heatxloop	raw	421.35	LeNet5 > KaNet5 by 421.35%	Yes (0.00)	F
heatxloop	norm	8.23	LeNet5 > KaNet5 by 8.23%	Yes (0.00)	F
heatxloop	abs_ordinal	-8.83	KaNet5 > LeNet5 by 8.83%	Yes (0.00)	F
heatxloop	zscore	39.69	LeNet5 > KaNet5 by 39.69%	Yes (0.00)	F
heatxorl_mask	raw	486.60	LeNet5 > KaNet5 by 486.60%	Yes (0.00)	F
heatxorl_mask	norm	4.92	LeNet5 > KaNet5 by 4.92%	Yes (0.00)	F
heatxorl_mask	abs_ordinal	6.00	LeNet5 > KaNet5 by 6.00%	Yes (0.00)	F
heatxorl_mask	zscore	59.80	LeNet5 > KaNet5 by 59.80%	Yes (0.00)	F
heatxvcr_mask	raw	579.93	LeNet5 > KaNet5 by 579.93%	Yes (0.00)	F
heatxvcr_mask	norm	-16.38	KaNet5 > LeNet5 by 16.38%	Yes (0.00)	F
heatxvcr_mask	abs_ordinal	7.30	LeNet5 > KaNet5 by 7.30%	Yes (0.00)	F
heatxvcr_mask	zscore	76.35	LeNet5 > KaNet5 by 76.35%	Yes (0.00)	F
segmentation_heat_loop	raw	0.00	LeNet5 = KaNet5	No (1.00)	F
segmentation_heat_loop	norm	0.00	LeNet5 = KaNet5	No (1.00)	F
segmentation_heat_loop	abs_ordinal	0.00	LeNet5 = KaNet5	No (1.00)	F
segmentation_heat_loop	zscore	0.00	LeNet5 = KaNet5	No (1.00)	F

Table 7.6: Grad-CAM Metrics: Standard-LeNet5 vs. KaNet5 (Effect@50, Wilcoxon)

Metric	Norm. type	Effect@50 (%)	Comparison@50	Informative?	Changed
heatxedge	raw	5.19	LeNet5 > KaNet5 by 5.19%	Yes (0.00)	T
heatxedge	norm	5.19	LeNet5 > KaNet5 by 5.19%	Yes (0.00)	T
heatxedge	abs_ordinal	-4.74	KaNet5 > LeNet5 by 4.74%	Yes (0.00)	T
heatxedge	zscore	5.19	LeNet5 > KaNet5 by 5.19%	Yes (0.00)	T
heatxcorner	raw	4.89	LeNet5 > KaNet5 by 4.89%	Yes (0.00)	T
heatxcorner	norm	4.89	LeNet5 > KaNet5 by 4.89%	Yes (0.00)	T
heatxcorner	abs_ordinal	-4.85	KaNet5 > LeNet5 by 4.85%	Yes (0.00)	T
heatxcorner	zscore	4.89	LeNet5 > KaNet5 by 4.89%	Yes (0.00)	T
heatxloop	raw	0.81	LeNet5 > KaNet5 by 0.81%	Yes (0.00)	T
heatxloop	norm	0.81	LeNet5 > KaNet5 by 0.81%	Yes (0.00)	T
heatxloop	abs_ordinal	-2.82	KaNet5 > LeNet5 by 2.82%	Yes (0.00)	T
heatxloop	zscore	0.81	LeNet5 > KaNet5 by 0.81%	Yes (0.00)	T
heatxorl_mask	raw	-8.24	KaNet5 > LeNet5 by 8.24%	Yes (0.00)	T
heatxorl_mask	norm	-8.24	KaNet5 > LeNet5 by 8.24%	Yes (0.00)	T
heatxorl_mask	abs_ordinal	-2.62	KaNet5 > LeNet5 by 2.62%	Yes (0.00)	T
heatxorl_mask	zscore	-8.24	KaNet5 > LeNet5 by 8.24%	Yes (0.00)	T
heatxvcr_mask	raw	6.04	LeNet5 > KaNet5 by 6.04%	Yes (0.00)	T
heatxvcr_mask	norm	6.04	LeNet5 > KaNet5 by 6.04%	Yes (0.00)	T
heatxvcr_mask	abs_ordinal	-1.21	KaNet5 > LeNet5 by 1.21%	Yes (0.00)	F
heatxvcr_mask	zscore	6.04	LeNet5 > KaNet5 by 6.04%	Yes (0.00)	T
segmentation_heat_loop	raw	0.00	LeNet5 = KaNet5	Yes (0.00)	F
segmentation_heat_loop	norm	0.00	LeNet5 = KaNet5	Yes (0.00)	F
segmentation_heat_loop	abs_ordinal	0.00	LeNet5 = KaNet5	No (1.00)	F
segmentation_heat_loop	zscore	0.00	LeNet5 = KaNet5	Yes (0.00)	F

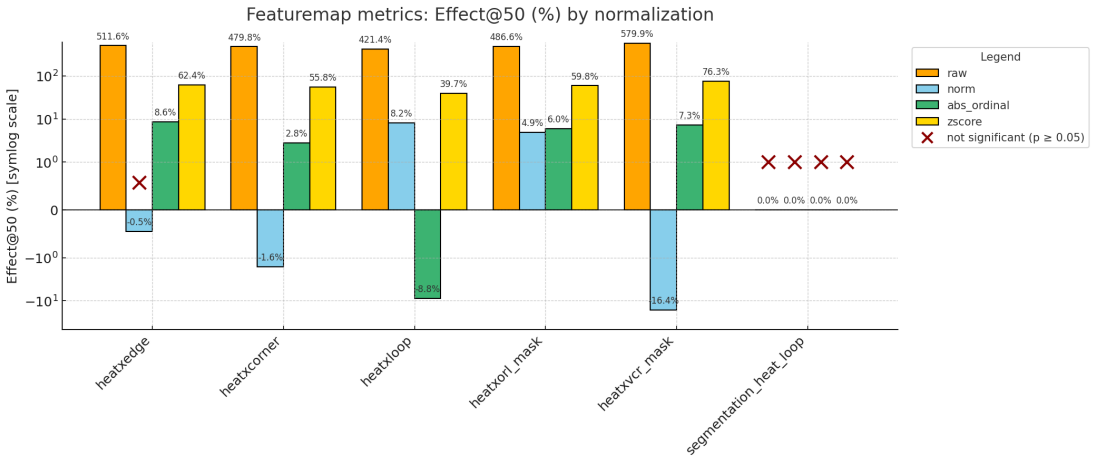


Figure 7.10: Feature-map metrics at epoch 50. Effect sizes (%) are shown across normalization schemes. Large raw and z-score effects reflect amplitude sensitivity in LeNet5, while norm and abs_ordinal highlight cases where KaNet5 gains relative advantages.

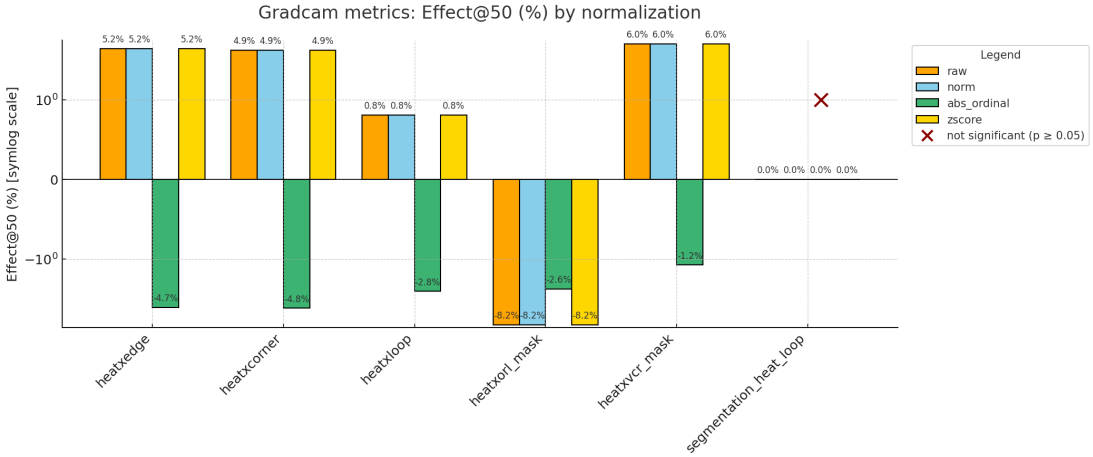


Figure 7.11: Grad-CAM metrics at epoch 50. Unlike feature maps, Grad-CAM captures discriminative evidence. LeNet5 shows small consistent gains on edges, corners, and vertical strokes, while KaNet5 achieves stronger alignment with concavity (ORL) and rank-based metrics.

7.3.8 Interpretative discussion

Sensitivity vs. Selectivity at Epoch 50

At epoch 50, a Wilcoxon signed-rank test (two-sided; H_0: median difference $= 0$) reveals a consistent bifurcation between *sensitivity* (activation magnitude) and *selectivity* (relative prioritization of salient regions).

Feature-Map Analysis

Feature-map analysis highlights a pronounced amplitude bias in **LeNet5**. Under *raw* and *z-score* normalizations, LeNet5 achieves extremely large percentage advantages across all structural families—edges, corners, loops, ORL, and VCR (e.g., `heatxvcr_mask`: +579.93% raw, +76.35% z). These inflated margins are partly explained by denominator effects: KaNet5 produces more negative activations, pushing its mean closer to zero and amplifying the relative difference.

Once scale effects are neutralized via *min–max norm* and *abs_ordinal*, local advantages often shift in favor of **KaNet5**. Examples include superior performance in *corner sensitivity* under norm (-1.62%, $p < .05$), *loop prioritization* under abs-ordinal (-8.83%, $p < .05$), and *vertical-structure sensitivity (VCR)* under norm (-16.38%, $p < .05$). By contrast, **LeNet5** retains modest strength in *loops* under norm ($+8.23\%$, $p < .05$) and continues to outperform in *ORL* under abs-ordinal ($+6.00\%$, $p < .05$).

Grad-CAM Analysis

Grad-CAM, which probes discriminative evidence rather than raw activation energy, provides a complementary perspective. Here, **LeNet5** maintains small but consistent intensity-based gains on *edges* and *corners* ($\approx +5\%$) and on *VCR* ($+6.04\%$) across raw, norm, and z-score. In contrast, **KaNet5** is systematically stronger on *ORL masks*, with significant gains across all transforms (-8.24% raw/norm/z; -2.62% abs-ordinal). It also achieves superior *ordinal ranking* on edges, corners, and loops ($\approx -3\%$ to -5%, $p < .05$). For *loop segmentation*, both models converge to parity (*Effect@50* $= 0$), indicating equivalent compactness of Grad-CAM activations at convergence.

Operational Implications

The operational implications appear clear. In *amplitude-centric pipelines*—such as calibration-free thresholding, saliency distillation, or anomaly scoring—**LeNet5** is the safer model, providing stronger signal margins without extensive fine-tuning. Conversely, in *ranking- or normalization-centric pipelines*—such as top-k region selection or budgeted inference—**KaNet5** is often more competitive and may be preferable, particularly when concavity cues (ORL) are critical. Where *vertical structure (VCR)* is pivotal, **LeNet5**'s discriminative edge supports its selection as the default choice.

Summary. Magnitude-sensitive settings favor **LeNet5**, rank-order and ORL-sensitive contexts favor **KaNet5**, and inflated raw-percentage differences should be interpreted cautiously as potential scale/sign artifacts rather than intrinsic architectural properties.

KaNet5 vs. LeNet5: Broader Perspective

Beyond this task, the evidence suggests that **KaNet5** and **LeNet5** optimize for different situations. **LeNet5**—a classic convolutional architecture—tends to maximize magnitude-driven *sensitivity*, which may benefit pipelines that depend on absolute signal strength or anomaly-style scoring. **KaNet5**, by contrast, appears to emphasize rank-consistent *selectivity* and stronger treatment of concavity cues (ORL), which may align better with workflows that privilege *relative prioritization* (e.g., top-k attention, budgeted inference) and require *auditable* salience ordering. It is plausible that KaNet5's compositional biases also support more controllable or interpretable behaviors in low-data or weakly supervised settings, although this remains a hypothesis pending broader validation.

Future Directions

Promising research directions include:

- **Hybrid designs** that combine *LeNet-style backbones* with *KaNet5 modules* (e.g., in heads or bottlenecks) for interpretable scoring—a path to retain LeNet5's amplitude margin while injecting KaNet5's ordinal priors.
- **Causal validation** of KaNet5's ordinal advantages (deletion/insertion, counterfactual masking) to test whether observed ranking gains translate into actionable evidence, not just correlational artifacts.
- **Domain robustness studies**, exploring whether **KaNet5** retains rank consistency under data scarcity and domain shift relative to **LeNet5**'s magnitude advantages; this is plausible but requires empirical confirmation.
- **Calibration and uncertainty**, assessing whether KaNet5's scores calibrate more reliably under normalization; current results suggest this could hold, but the claim is hypothetical beyond the present dataset.
- **Efficiency trade-offs**, clarifying when **KaNet5** should replace a LeNet5-style baseline and when it should augment it to deliver both signal amplitude and decision-level transparency, under real parameter and latency budgets.

Chapter 8

Conclusions

Overall assessment

This thesis set out to compare classical convolutional and fully connected neural architectures with their Kolmogorov–Arnold inspired counterparts. The empirical evidence collected in Chapters 5–7 demonstrates that, on both MNIST and the Extended MNIST dataset, the expressive power endowed by edge–wise spline functions does not translate into a systematic performance advantage over standard weight–based models. In the fully connected regime, LeNet300 and KaNet300 achieve virtually identical classification accuracy and loss, and the results remain indistinguishable even after ablating hidden layers. In the convolutional setting, LeNet5 consistently slightly surpasses KaNet5 in both training and test accuracy, even under different regularisation regimes. These findings suggest that, at least on handwritten character recognition tasks, the additional functional flexibility of KAN weights offers no clear gain in generalisation.

Beyond accuracy, the thesis developed a battery of interpretability analyses. Feature-map and Grad-CAM visualisations reveal that both architectures attend to meaningful parts of the input, but with notable differences. LeNet5 tends to distribute attention more uniformly across the character structure, generating heat maps with greater overall intensity. KaNet5 produces sparser, more localised patterns: its Grad-CAM maps emphasise specific strokes and concavities, and its feature-map activations often show reduced magnitude but higher variability. A quantitative comparison via structural attention metrics confirms this dichotomy. Under raw or z-score normalisations, LeNet5 dominates due to its stronger activations; when heat maps are ranked, KaNet5 enjoys modest advantages on corner detection and concavity alignment. These results highlight that LeNet5 excels in sensitivity (large responses), whereas KaNet5 is competitive in selectivity (relative ranking of salient regions).

Taken together, the experiments offer a nuanced picture. KANs remain a principled and interpretable alternative to classical networks, but their benefits are context dependent. On simple image-classification tasks they do not outperform standard CNNs, and the choice between architectures depends more on downstream requirements (e.g., prioritising absolute activation magnitude versus

ranked salience) than on accuracy.

Future research directions

While the current study focused on handwritten character recognition, the framework developed here opens numerous avenues for future work. The following directions are particularly promising:

- **Extending to diverse datasets.** Testing both architectures on larger and more varied datasets—such as ImageNet, COCO or domain-specific collections—would reveal whether the sensitivity–selectivity trade-off observed on EMNIST persists in more complex settings. The use of EMNIST, which contains only a single object per image, may have limited the benefits of ConvKAN's pinpointing capability; datasets containing multiple objects could highlight its strength in localising features. Li *et al.*[10] and Jamali *et al.*[11] report promising results of KANs on medical and hyperspectral images, suggesting that broader validation is worthwhile.

- **Exploring new vision tasks.** Applying the interpretability pipeline to tasks beyond classification—such as object detection, semantic segmentation or scene understanding—could elucidate the roles of sensitivity and selectivity in more complex contexts. Recent work on remote-sensing and crop-field segmentation indicates that localised activations can enhance performance in multi-object environments[12, 13].

- **Hybrid architectures.** The pronounced pinpointing ability of ConvKAN invites exploration of hybrid designs that combine classical CNN backbones with KAN modules or attention mechanisms. Such hybrids might yield networks that retain high activation energy where needed while exploiting KANs' ability to focus on fine-grained structures.

- **Advanced interpretability techniques.** Future studies could integrate the current structural metrics with complementary approaches—such as multi-layer saliency mapping, feature inversion or localised relevance propagation—to build a more comprehensive picture of how KANs process information. Combining multiple interpretability tools may uncover subtler differences between architectures.

- **Segmentation and dense prediction.** Given ConvKAN's localisation strength, investigating its performance on segmentation tasks is a natural extension. Early results by Wang *et al.*[14] show that KAN-based models improve flood-extent extraction, hinting that focused activations could enhance boundary detection in medical or satellite image segmentation.

These directions aim not only to improve model performance but also to deepen our understanding of the internal mechanisms driving interpretability

in neural networks. By extending KANs beyond simple classification, exploring hybrid designs and enriching the interpretability toolbox, future research can better harness the unique inductive biases inspired by the Kolmogorov–Arnold theorem.

Bibliography

[1] Simon Haykin. *Neural Networks and Learning Machines*. 3rd ed. Pearson Education, 2009.

[2] Boris Hanin. "Universal Function Approximation by Deep Neural Nets with Bounded Width and ReLU Activations". In: *Mathematics* 7.10 (2019). Submitted 29 September 2019 / Revised 15 October 2019 / Accepted 16 October 2019 / Published 18 October 2019. DOI: 10.3390/math7100992. URL: https://www.mdpi.com/2227-7390/7/10/992.

[3] Yann LeCun et al. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791.

[4] Ziming Liu et al. "KAN: Kolmogorov–Arnold Networks". In: *Proceedings of the International Conference on Learning Representations (ICLR)*. 2025, pp. 70367–70413. URL: https://proceedings.iclr.cc/paper_files/paper/2025/file/afaed89642ea100935e39d39a4da602c-Paper-Conference.pdf.

[5] Federico Girosi and Tomaso Poggio. "Representation Properties of Networks: Kolmogorov's Theorem Is Irrelevant". In: *Neural Computation* 1.4 (Dec. 1989), pp. 465–469. ISSN: 0899-7667. DOI: 10.1162/neco.1989.1.4.465.

[6] Gregory Cohen et al. "EMNIST: Extending MNIST to Handwritten Letters". In: *arXiv* (2017). DOI: 10.48550/arXiv.1702.05373. arXiv: 1702.05373. URL: https://arxiv.org/abs/1702.05373.

[7] Alexander Dylan Bodner et al. "Convolutional Kolmogorov–Arnold Networks". In: *arXiv* (2024). DOI: 10.48550/arXiv.2406.13155. arXiv: 2406.13155. URL: https://arxiv.org/abs/2406.13155.

[8] Tom Lyche and Knut Mørken. *Spline Methods*. Cambridge University Press, 2008. DOI: 10.1007/978-3-319-67885-6.

[9] Ramprasaath R. Selvaraju et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2017, pp. 618–626. DOI: 10.1109/ICCV.2017.74.

[10] Chenxin Li et al. “U-KAN Makes Strong Backbone for Medical Image Segmentation and Generation”. In: *arXiv* (2024). DOI: 10.48550/arXiv.2406.02918. arXiv: 2406.02918. URL: https://arxiv.org/abs/2406.02918.

[11] Ali Jamali et al. “How to Learn More? Exploring Kolmogorov–Arnold Networks for Hyperspectral Image Classification”. In: *Remote Sensing* 16.21 (2024). DOI: 10.3390/rs16214015. URL: https://www.mdpi.com/2072-4292/16/21/4015.

[12] X. Ma et al. “Kolmogorov–Arnold Network for Remote Sensing Image Semantic Segmentation”. In: *arXiv* (2025). DOI: 10.48550/arXiv.2501.07390. arXiv: 2501.07390. URL: https://arxiv.org/abs/2501.07390.

[13] D. R. Cambrin et al. “KAN You See It? KANs and Sentinel for Effective and Explainable Crop Field Segmentation”. In: *arXiv* (2024). DOI: 10.48550/arXiv.2408.07040. arXiv: 2408.07040. URL: https://arxiv.org/abs/2408.07040.

[14] C. Wang, X. Zhang, and L. Liu. “FloodKAN: Integrating Kolmogorov–Arnold Networks for Efficient Flood Extent Extraction”. In: *Remote Sensing* 17.4 (2025). DOI: 10.3390/rs17040564. URL: https://doi.org/10.3390/rs17040564.